

Gradient descent



10-701 Introduction to Machine Learning
Geoff Gordon

thanks to Matt Gormley and Henry Chai for some content

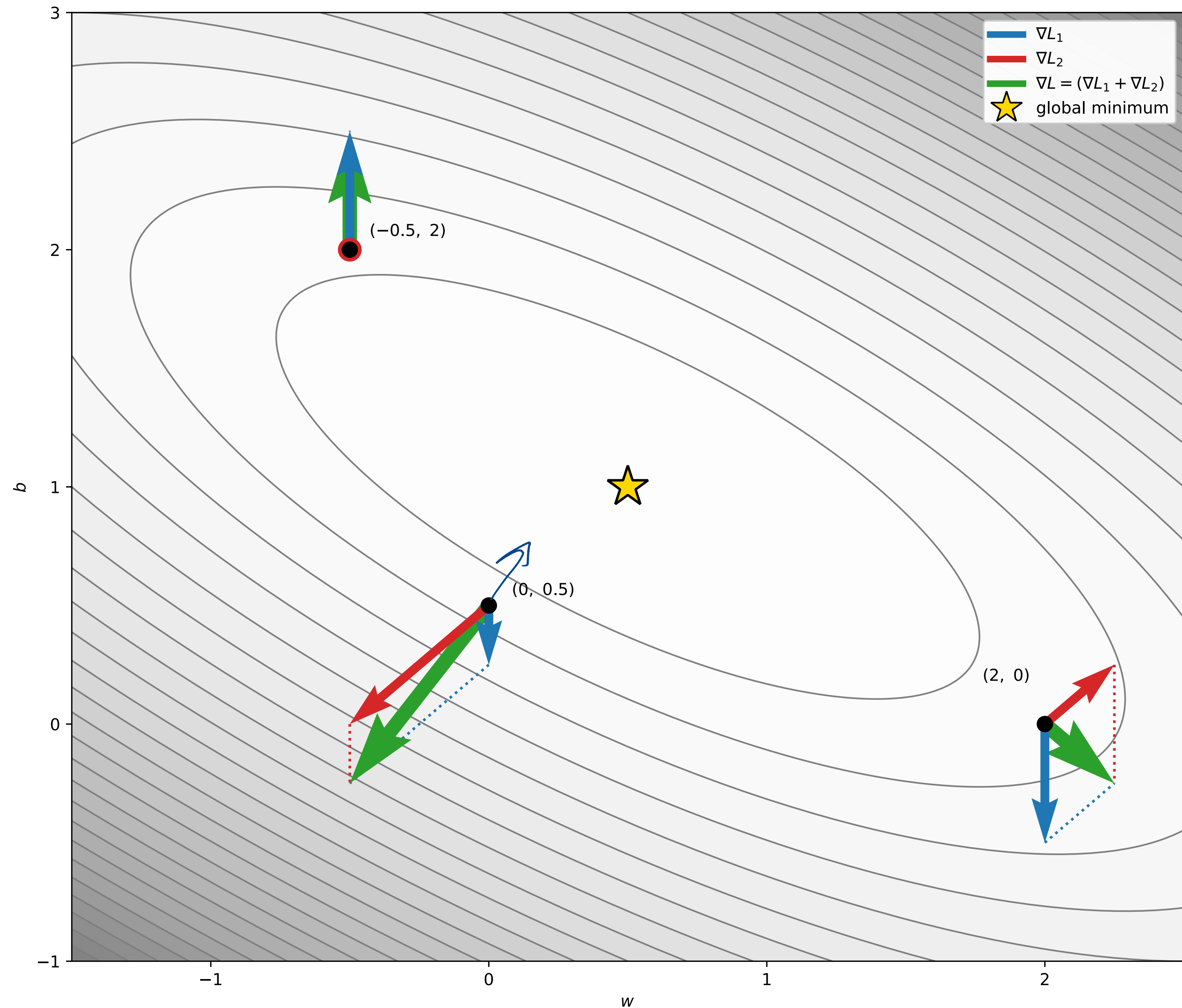
Scaling to big data

- When number of examples N is large, both gradient descent and exact method are slow
 - ▶ $O(Nk)$ per iteration vs. $O(Nk^2)$ total
- Need an algorithm with better dependence on N

Gradient is a sum

Not just that, but each term in the sum is iid

$$L(w, b) = \frac{1}{2}[(b - 1)^2 + (w + b - \frac{3}{2})^2]$$



One random example is enough

- Given a sum $L(\theta) = L_1(\theta) + \dots + L_N(\theta)$
- Multiply both sides by $\frac{1}{N}$:
 - ▶ $\frac{1}{N}L(\theta) = \frac{1}{N}L_1(\theta) + \dots + \frac{1}{N}L_N(\theta)$
- RHS has the form of an expectation:
 - ▶ sample an index k uniformly from $\{1 : N\}$, take $L_k(\theta)$
 - ▶ $\frac{1}{N}L(\theta) = \mathbb{E}_{k \sim \text{unif}}(L_k(\theta))$
- By linearity of expectation, gradient is:
 - ▶ $\frac{1}{N} \nabla L(\theta) = \mathbb{E}_{k \sim \text{unif}}(\nabla L_k(\theta))$
- That is: if we go in the direction suggested by *a single random* training example, then we *on average* go in the correct direction for the entire training set

What happens if we use just one example?

- If we just use one gradient sample, we can start improving very quickly: even before GD has made a single step
- But we have to deal with sampling noise:
 - ▶ individual steps might go in the wrong direction or off to the side
 - ▶ have to reduce step size η to compensate (true gradient changes slowly, so noise in successive samples has a chance to cancel)

Stochastic gradient descent

Algorithm: stochastic gradient descent w/ momentum

Inputs: loss function $L(\theta)$, initial value $\theta^{(1)}$, step size η , momentum β

$\bar{g} \leftarrow 0$

For $i \leftarrow 1, 2, \dots$:

if **converged** break

$k \leftarrow \text{Uniform}(\{1 : N\})$

$g \leftarrow \nabla L_k(\theta^{(i)}) \quad \leftarrow \text{for linear/ridge regression, } \frac{\lambda}{N}w + X^{(k)}(X^{(k)} \cdot w - Y^{(k)})$

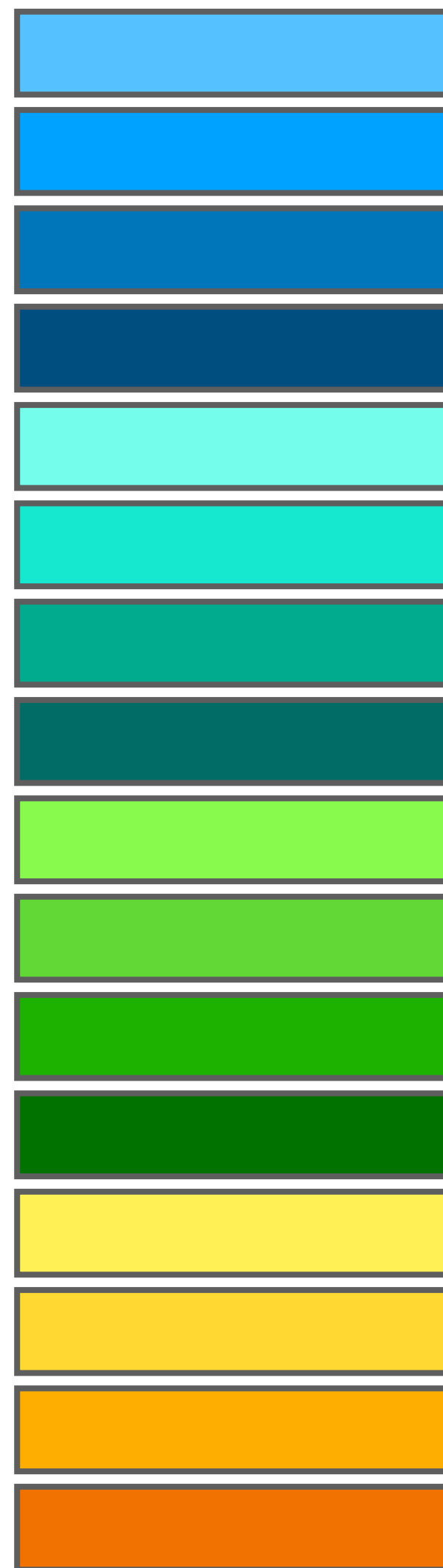
$\bar{g} \leftarrow \beta \bar{g} + (1 - \beta)g$

$\theta^{(i+1)} \leftarrow \theta^{(i)} - \eta \bar{g} \quad \leftarrow \text{we have absorbed a factor } N \text{ into learning rate}$
return $\theta^{(i)}$ (not a universal convention!)

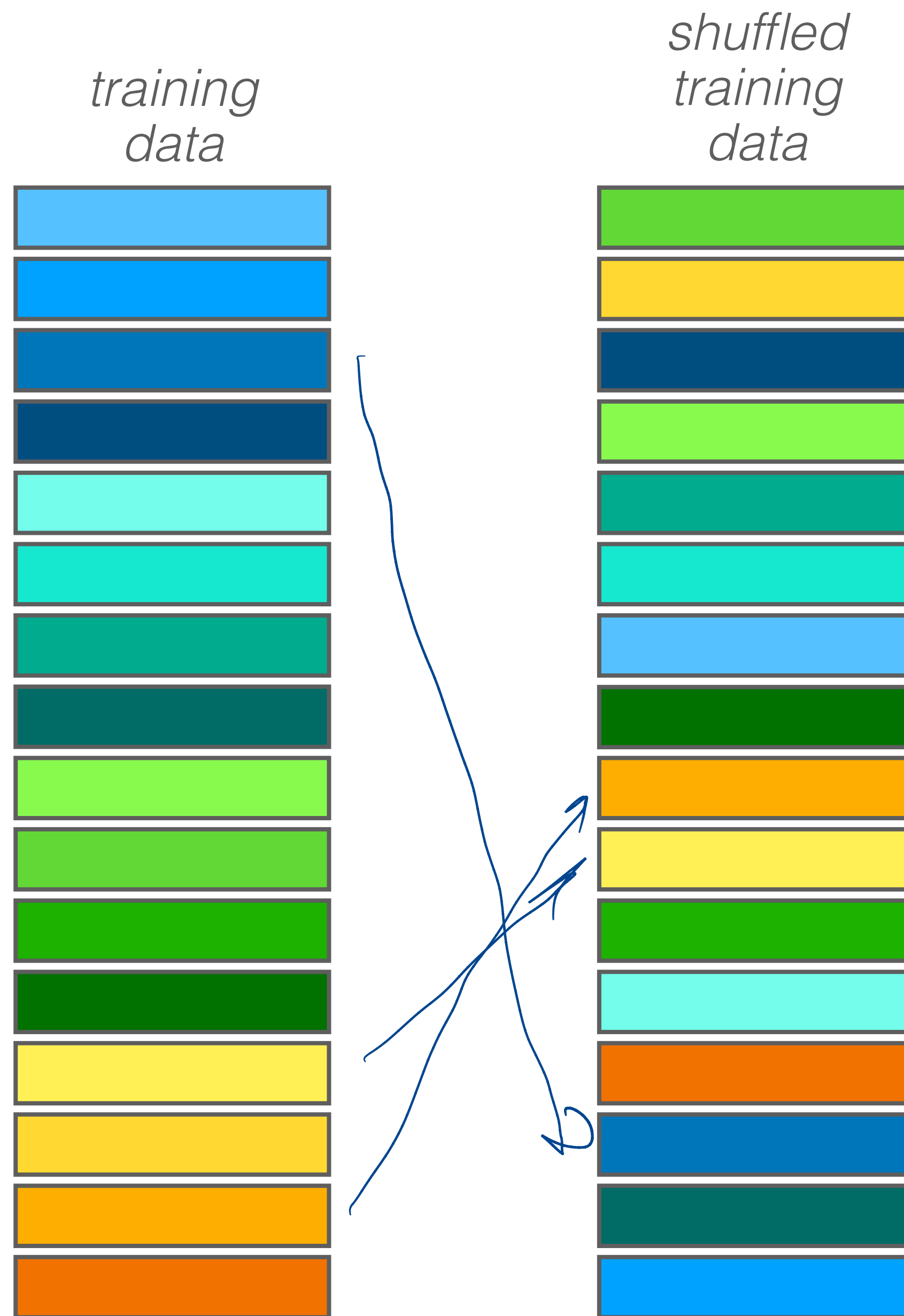
- If ridge $\lambda > 0$, common to split this term among all examples
 - ▶ pulls w back toward 0 — called **weight decay** *can exempt bias from decay*
- $\beta = 0$ gives SGD w/o momentum
- We typically write (and analyze) $k \leftarrow \text{Uniform}(\{1 : N\})$, but more commonly we shuffle and scan through entire training set (an **epoch**), then repeat — works noticeably better

Epochs

*training
data*

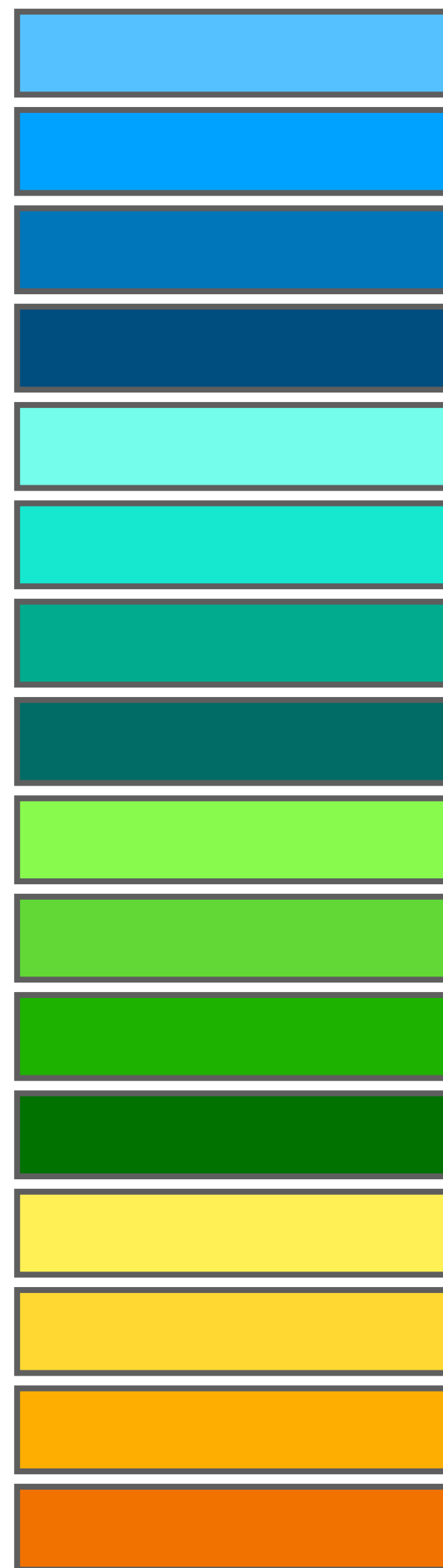


Epochs



Epochs

*training
data*



*shuffled
training
data*



iter 1

iter 2

iter 3

iter 4

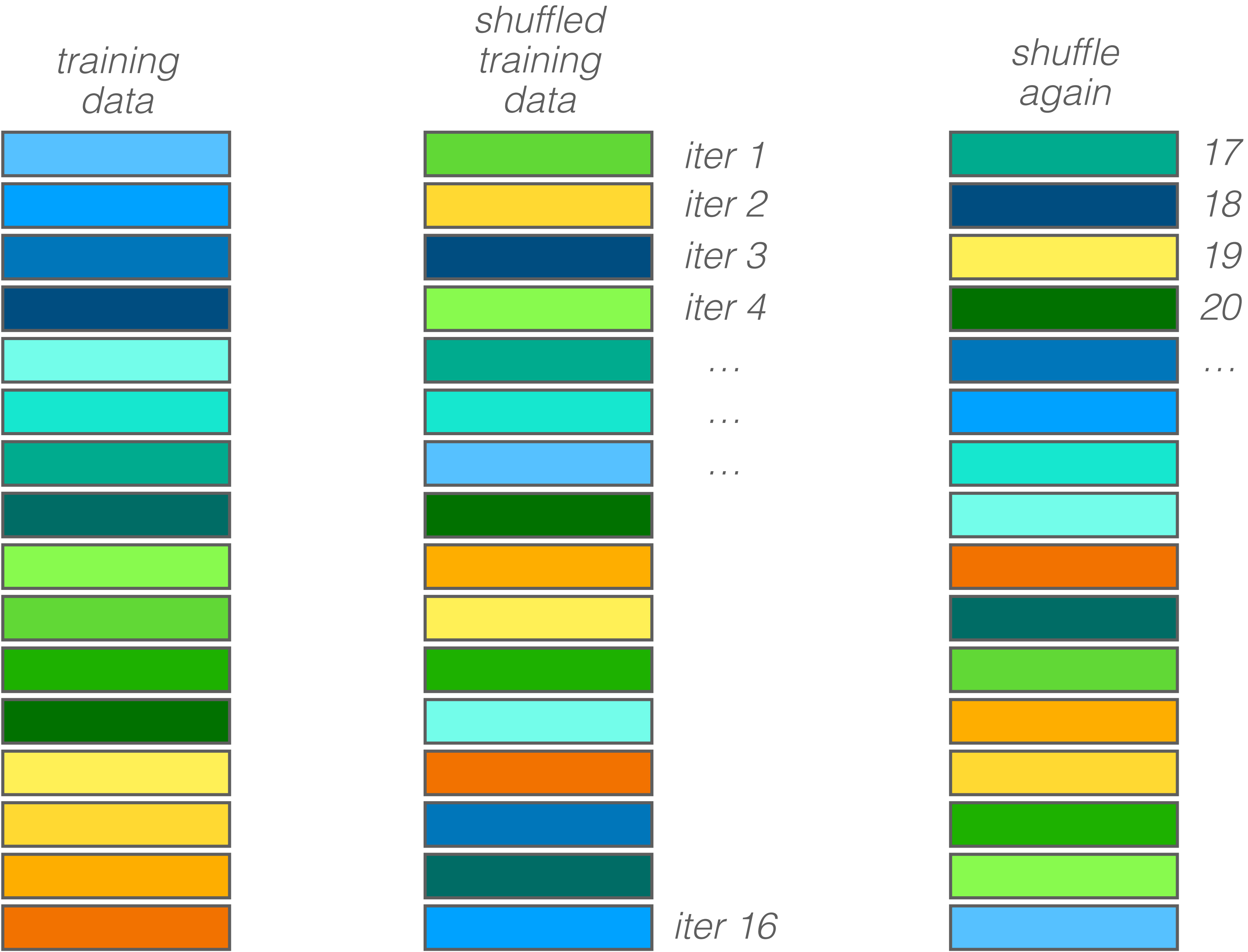
...

...

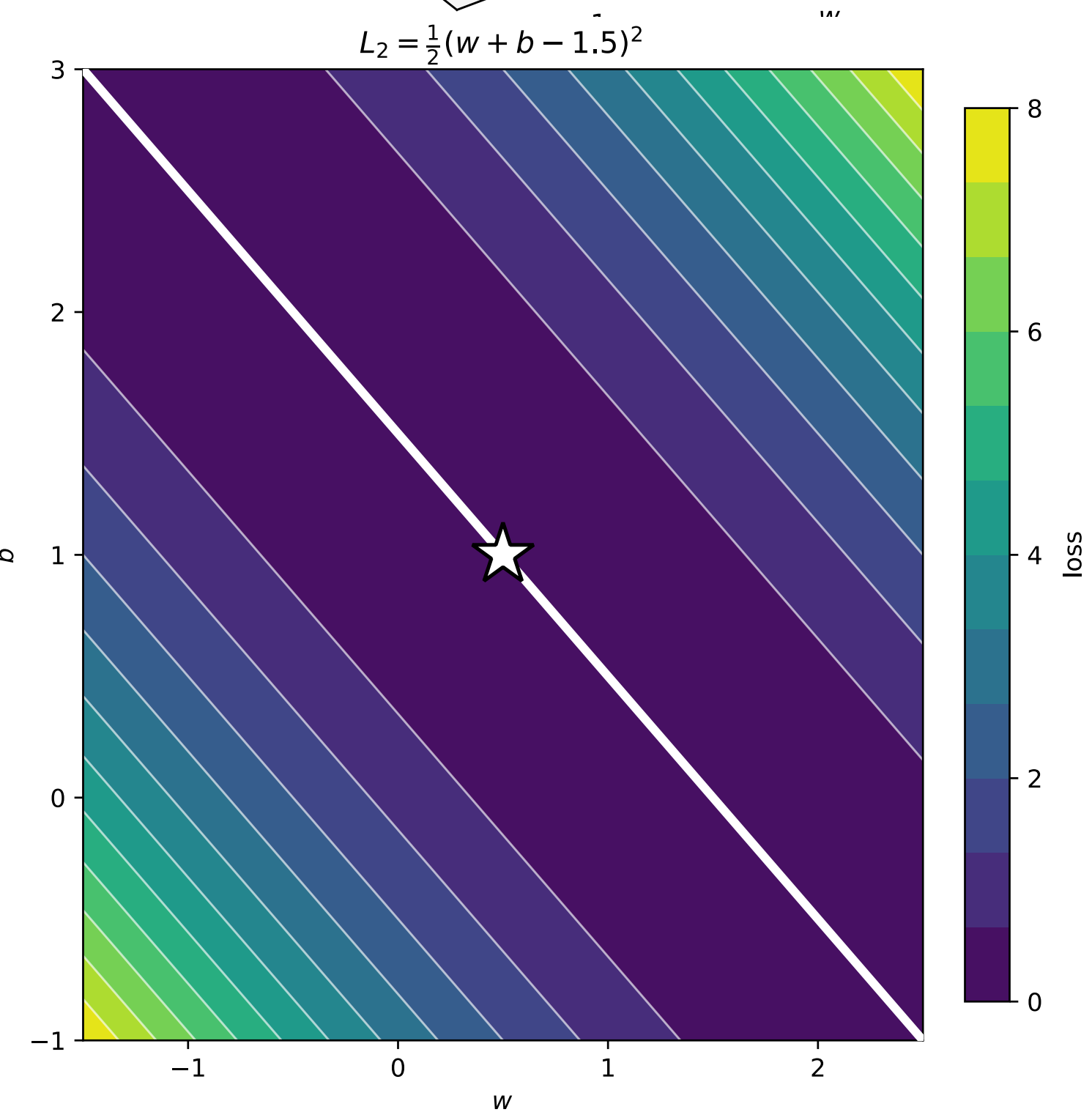
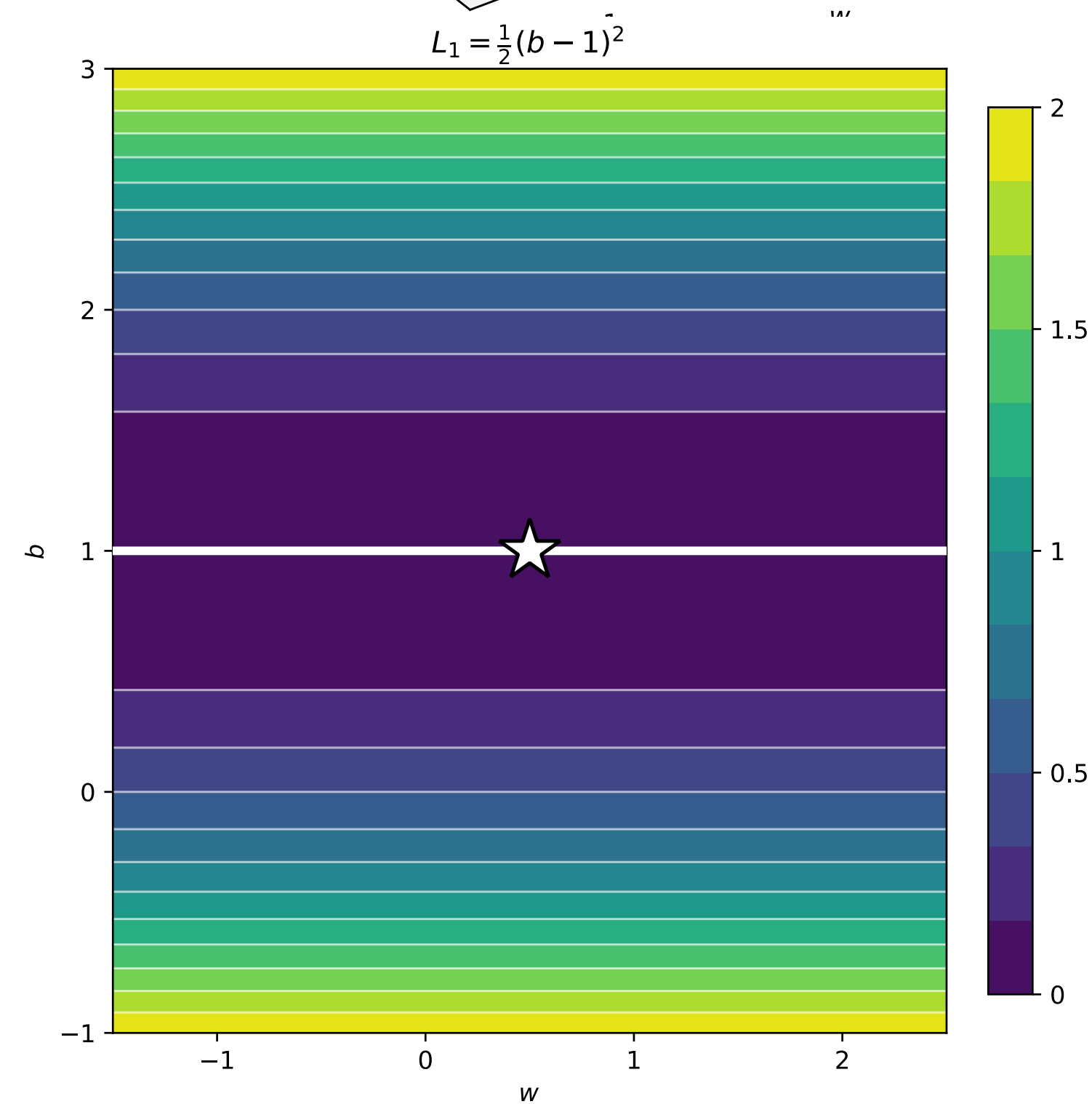
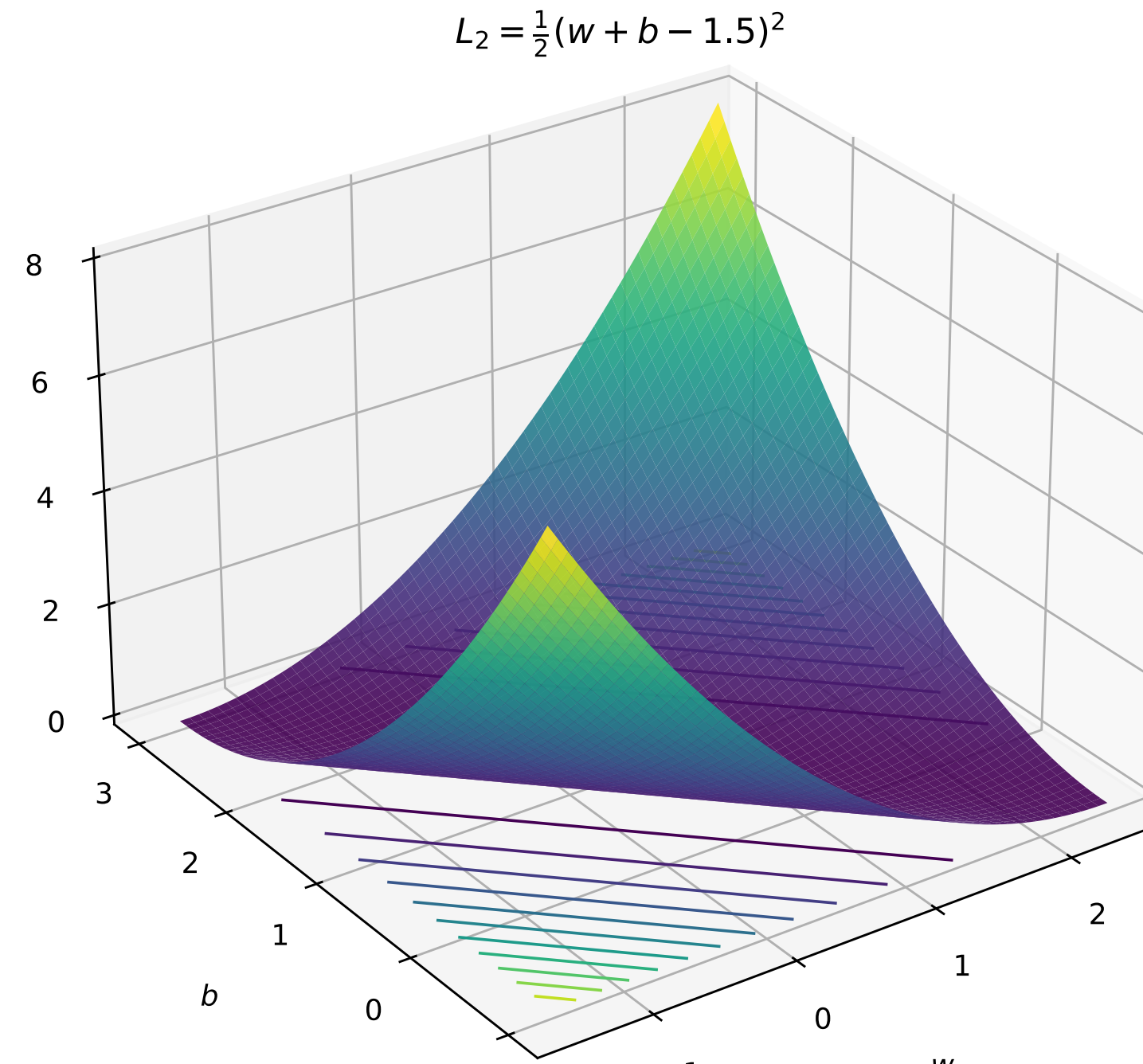
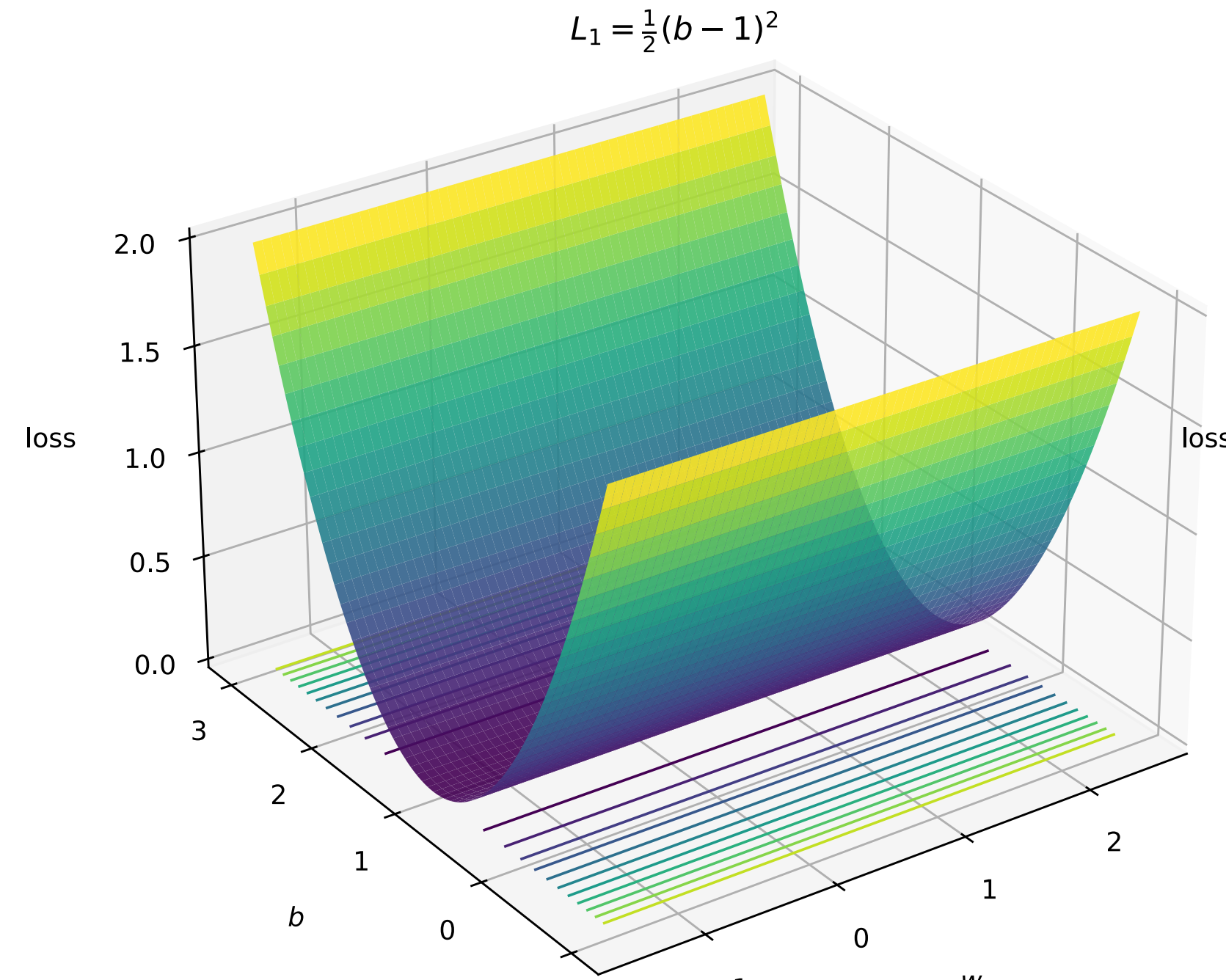
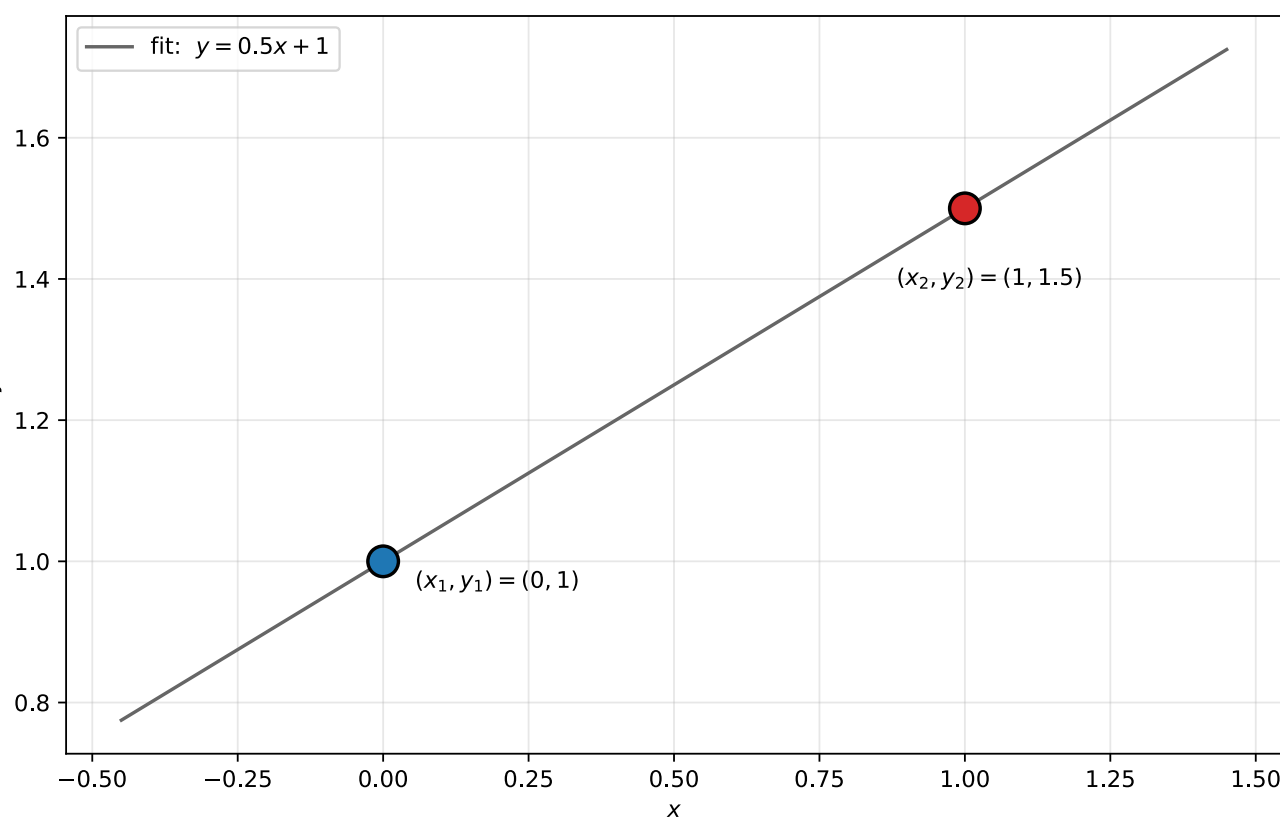
...

iter 16

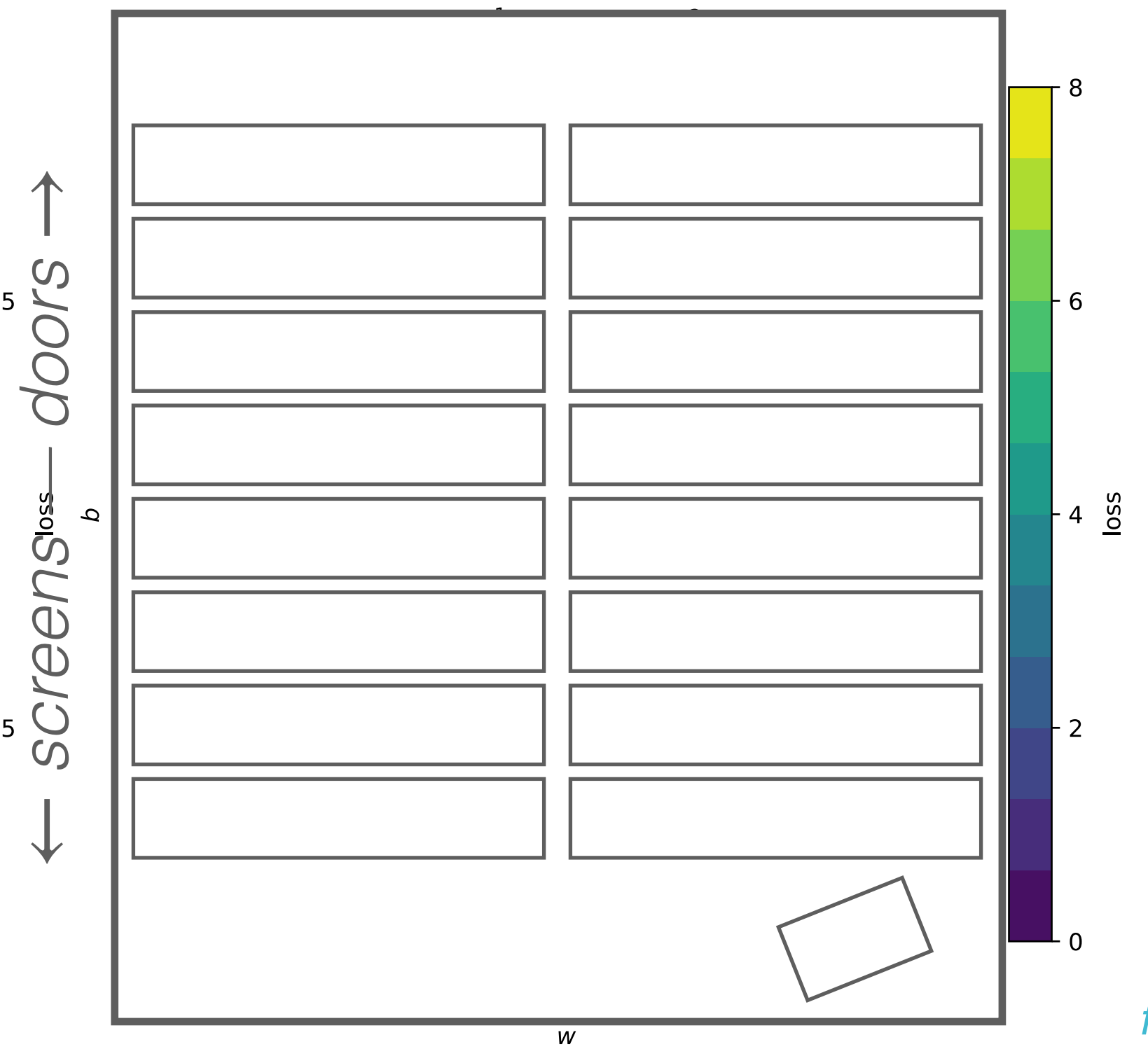
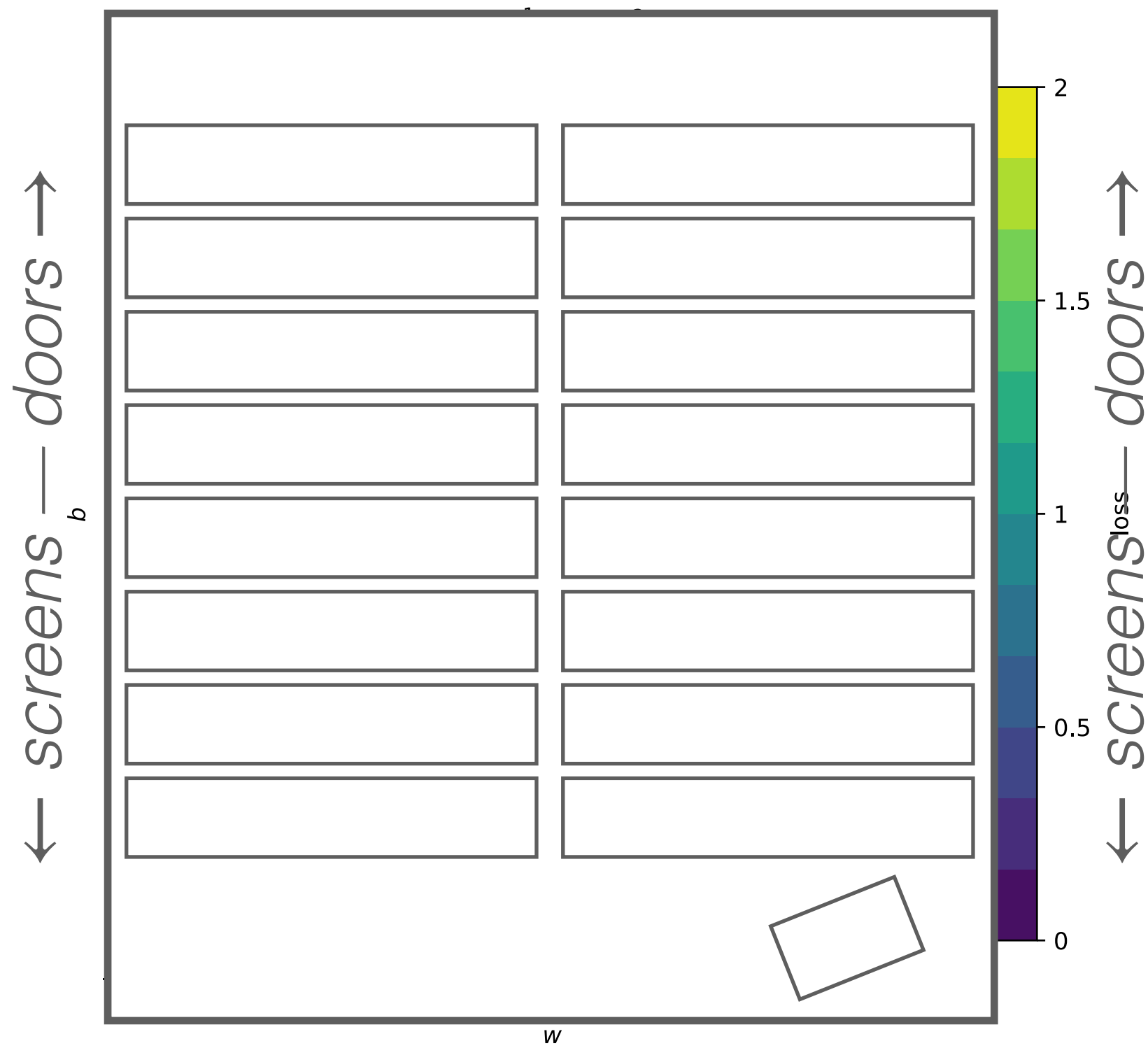
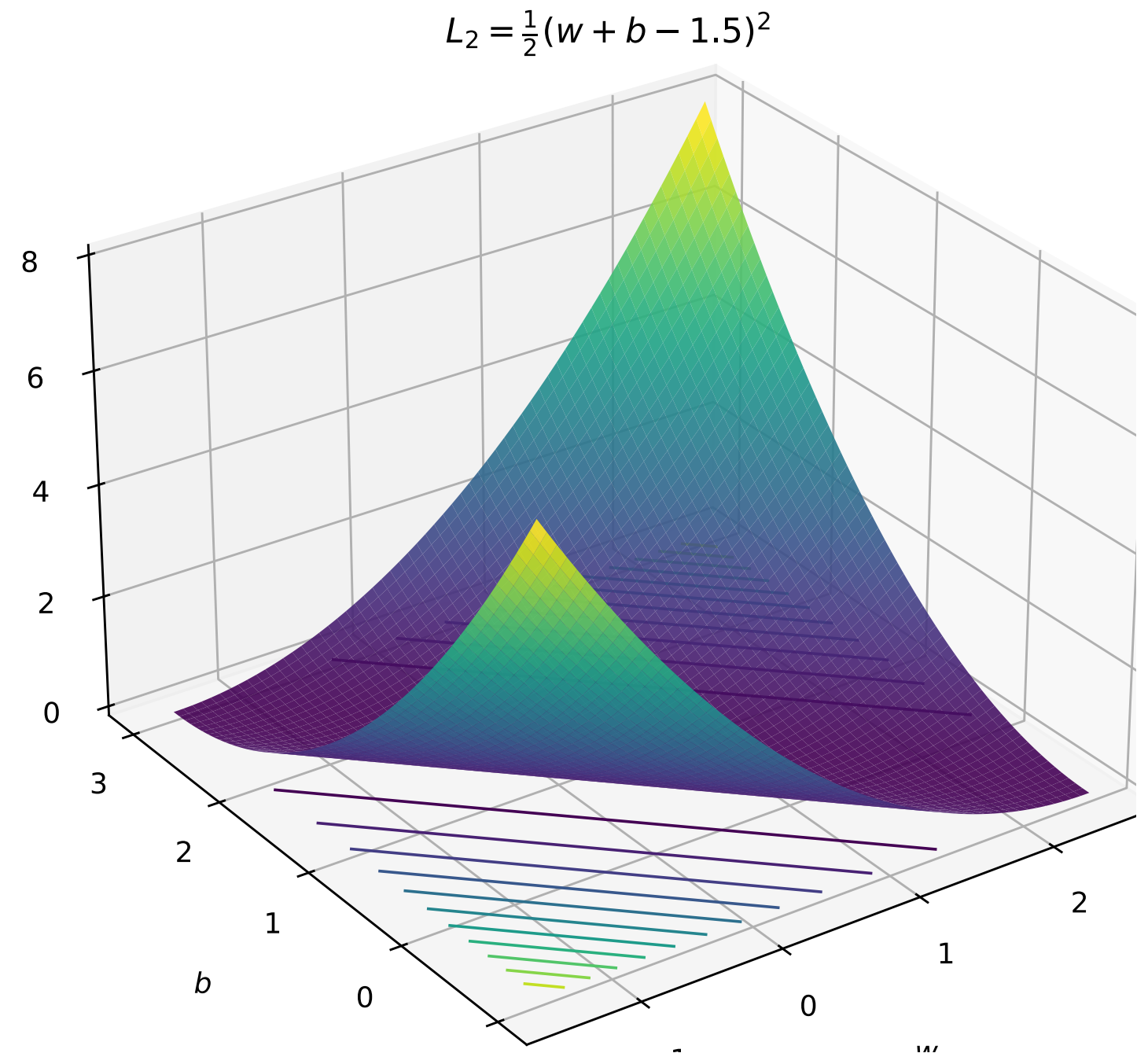
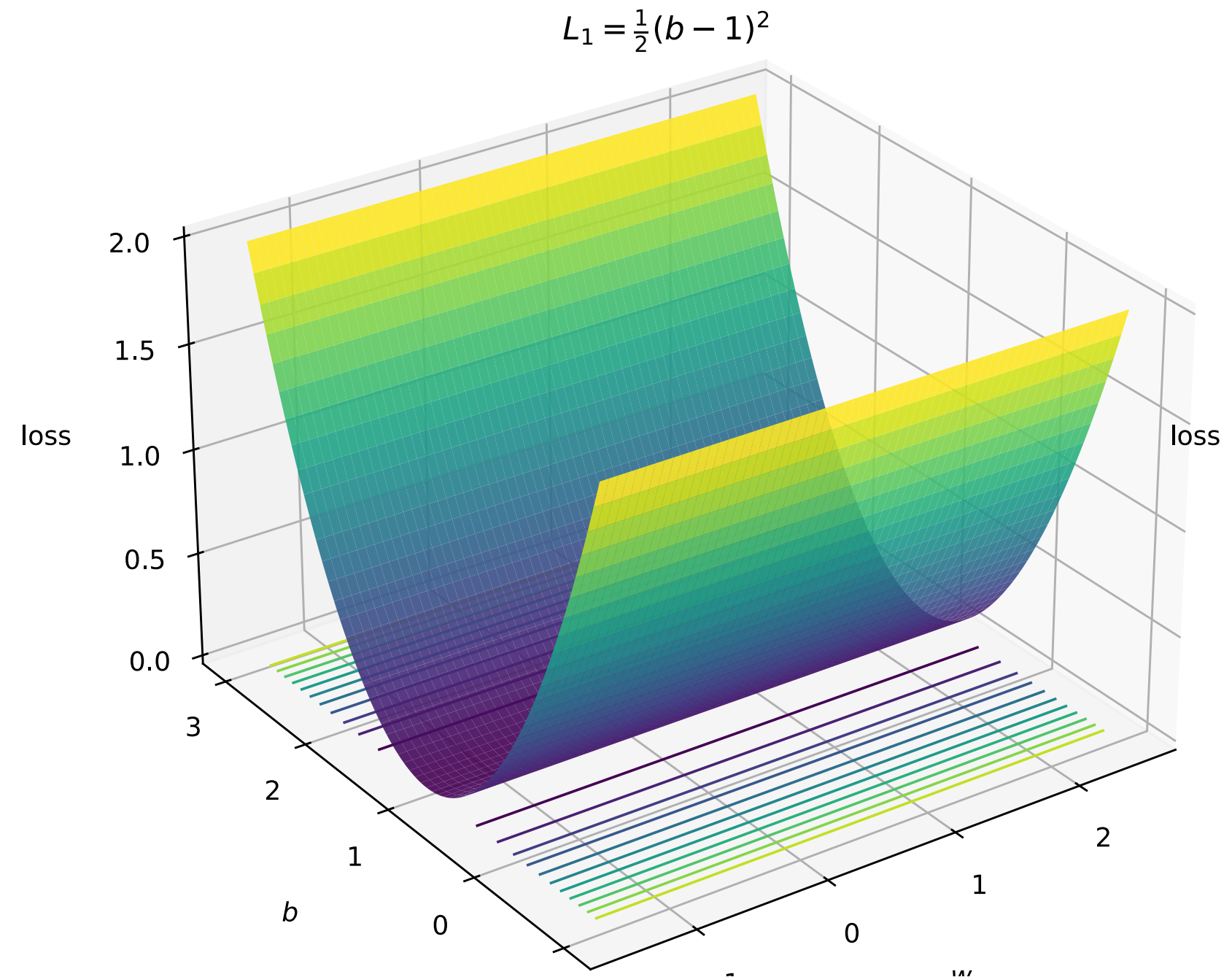
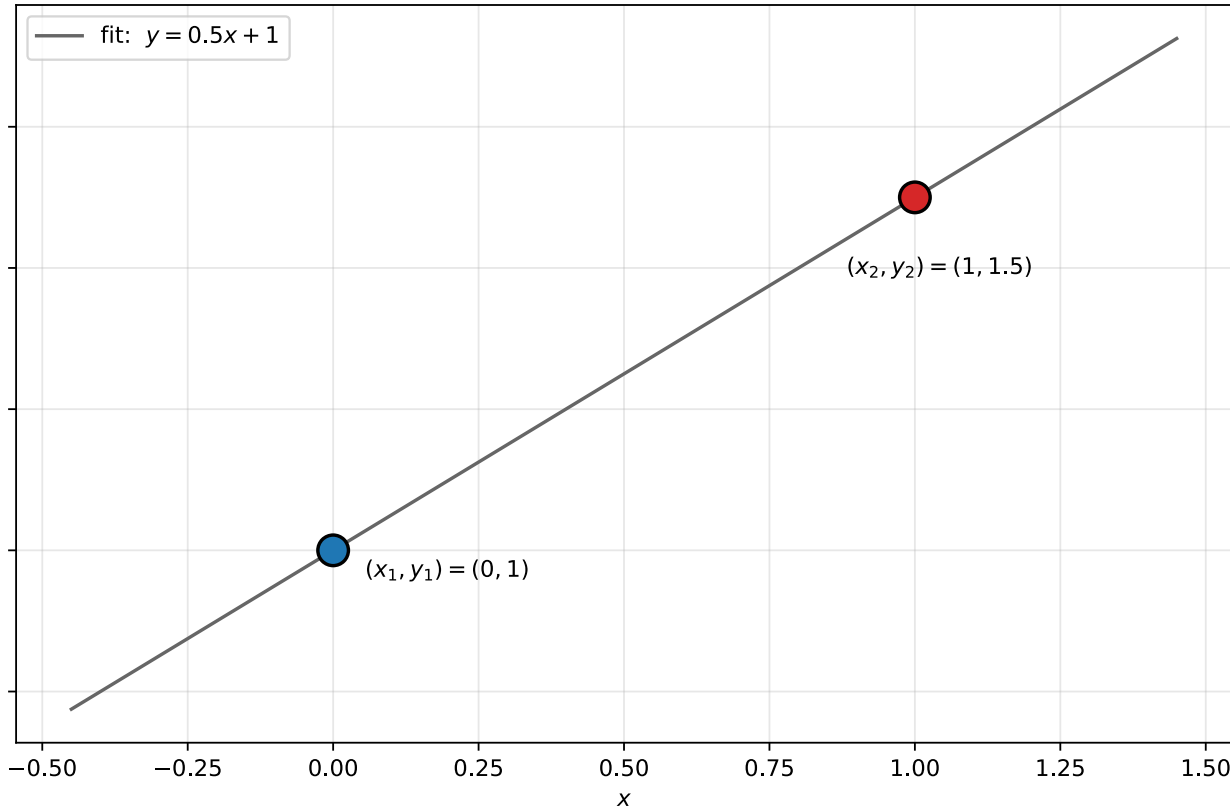
Epochs



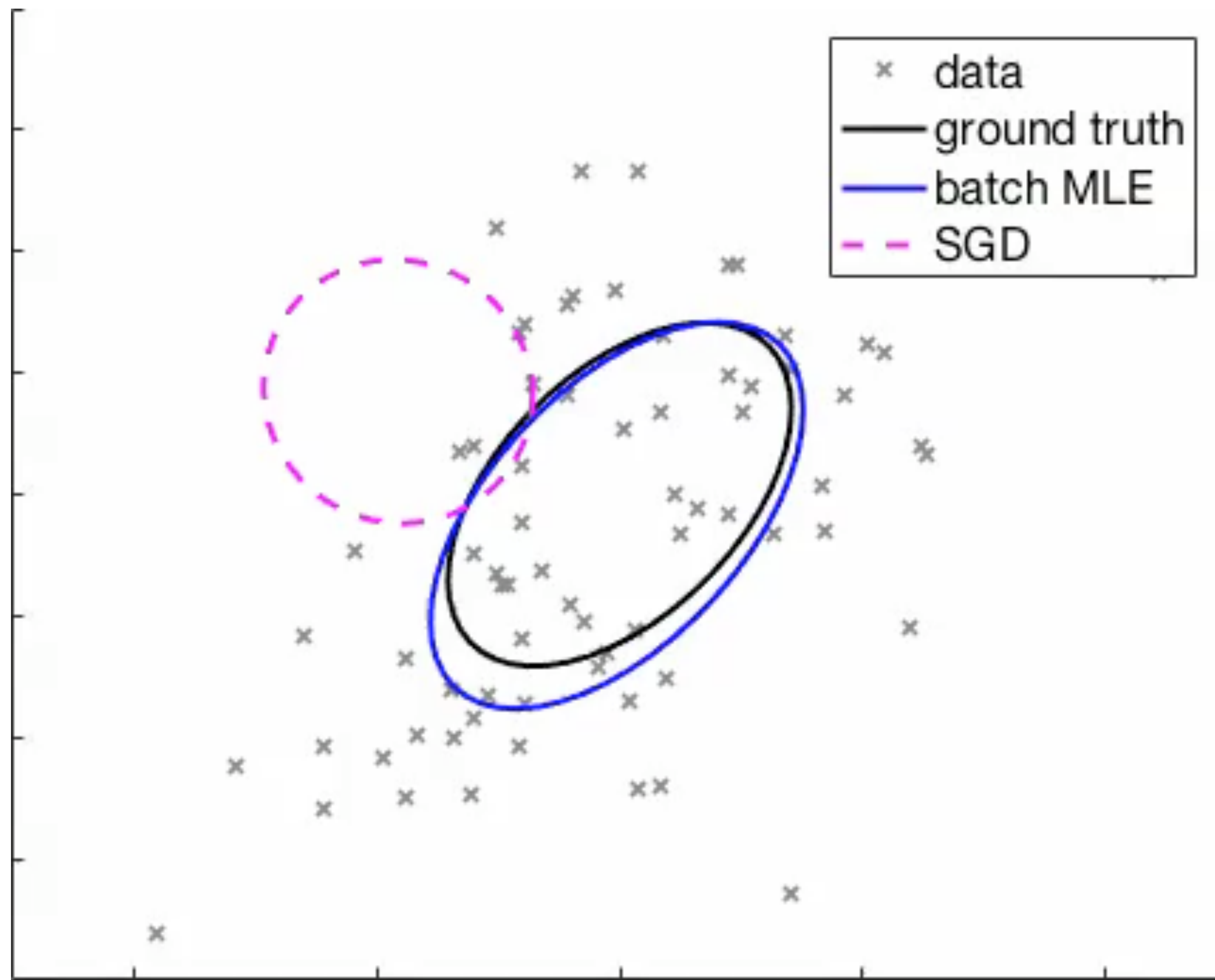
SGD



SGD



SGD example



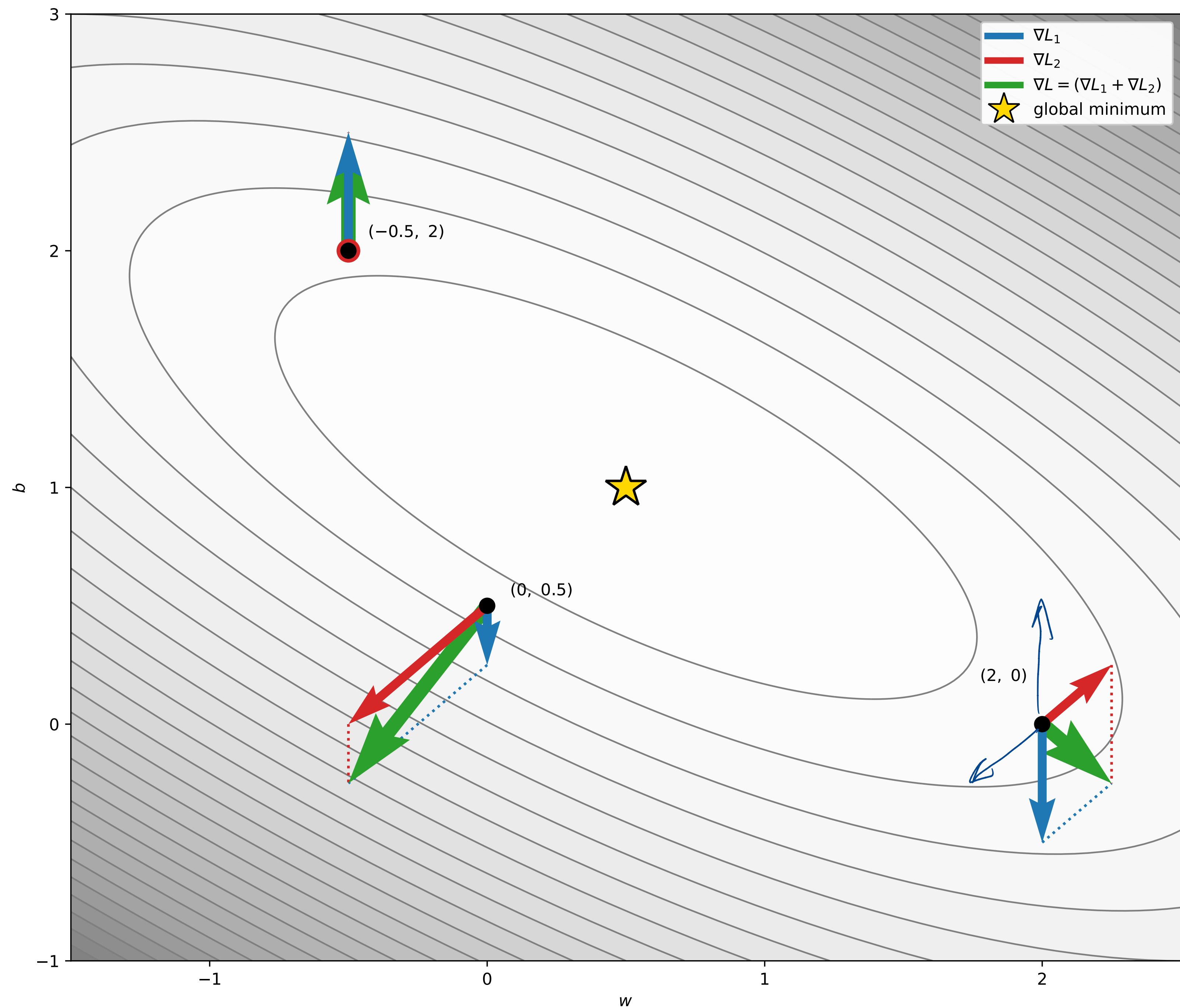
SGD phases

(not exact: soft transitions)

- Descent phase: gradient is large compared to sample noise
 - can happen if we're far from optimum
 - steady progress toward lower loss
 - larger learning rate is best (limited only by curvature of loss)
- Stochastic phase: gradient and noise are comparable magnitude
 - often happens as we get closer to optimum, or we might start in this phase
 - progress depends on keeping learning rate low enough to benefit from averaging
- Wandering phase: noise dominates gradient
 - little further progress in reducing loss, random walk in neighborhood of optimum — **noise floor**
 - can go back to stochastic phase by dropping learning rate and/or increasing batch size

Curvature limit

Curvature now places a sharper limit on learning rate: both noise and average curve can increase loss



A problem

- We can parallelize GD by splitting the training set among processors (cluster machines or GPU cores)
- Doesn't work for SGD (why?)
- Naively, SGD can't run fast on a cluster or GPU

Minibatch SGD

Algorithm: minibatch SGD w/ momentum

Inputs: loss function $L(\theta)$, initial value $\theta^{(1)}$, step size η , momentum β

$\bar{g} \leftarrow 0$

For $i \leftarrow 1, 2, \dots$:

if **converged** break

$K \leftarrow \text{Uniform}(\{1 : N\}, B)$ $\leftarrow$ the set K (w/ B elements) is the minibatch

$g \leftarrow \frac{1}{B} \sum_{k \in K} \nabla L_k(\theta^{(i)})$ $\leftarrow$ same expectation but averages out noise

$\bar{g} \leftarrow \beta \bar{g} + (1 - \beta)g$

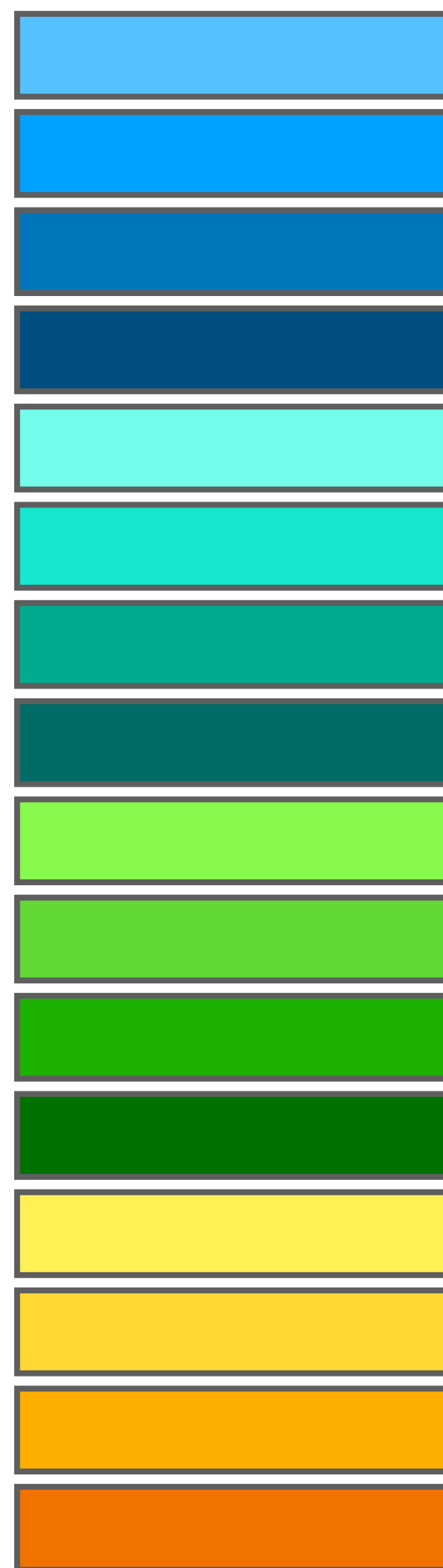
$\theta^{(i+1)} \leftarrow \theta^{(i)} - \eta \bar{g}$ $\leftarrow$ we still absorb a factor N into learning rate

return $\theta^{(i)}$

- Averaging B gradients reduces noise, lets us increase η
- A big enough minibatch can mostly-saturate a GPU
 - ▶ have to get fancier for a cluster since communication delays are much worse — e.g., asynchronous methods
- As before, shuffle and scan through data in each epoch

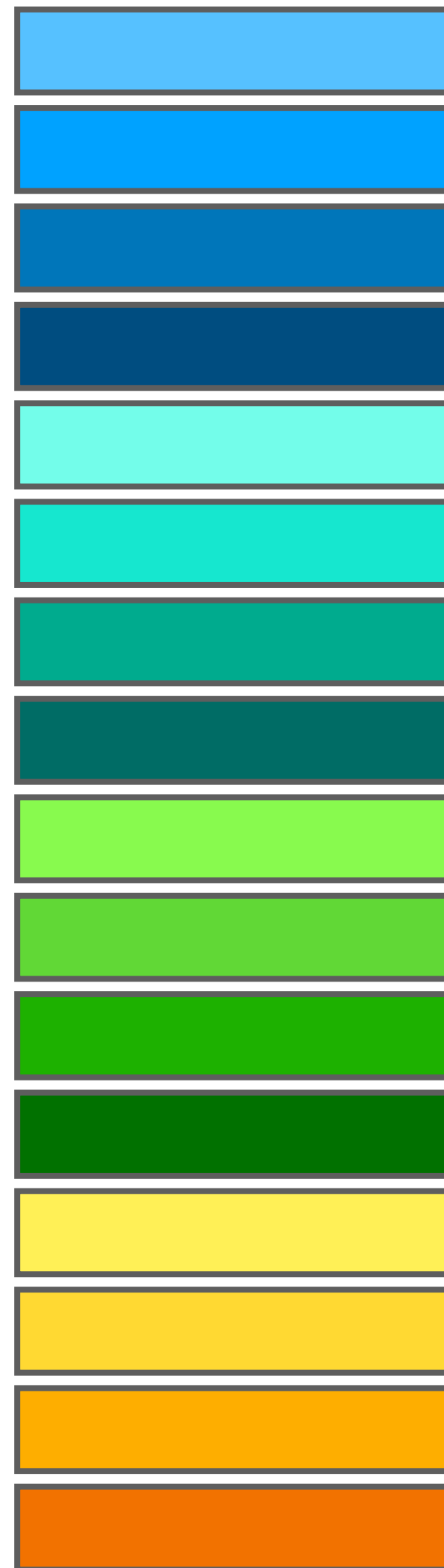
Epochs

*training
data*



Epochs

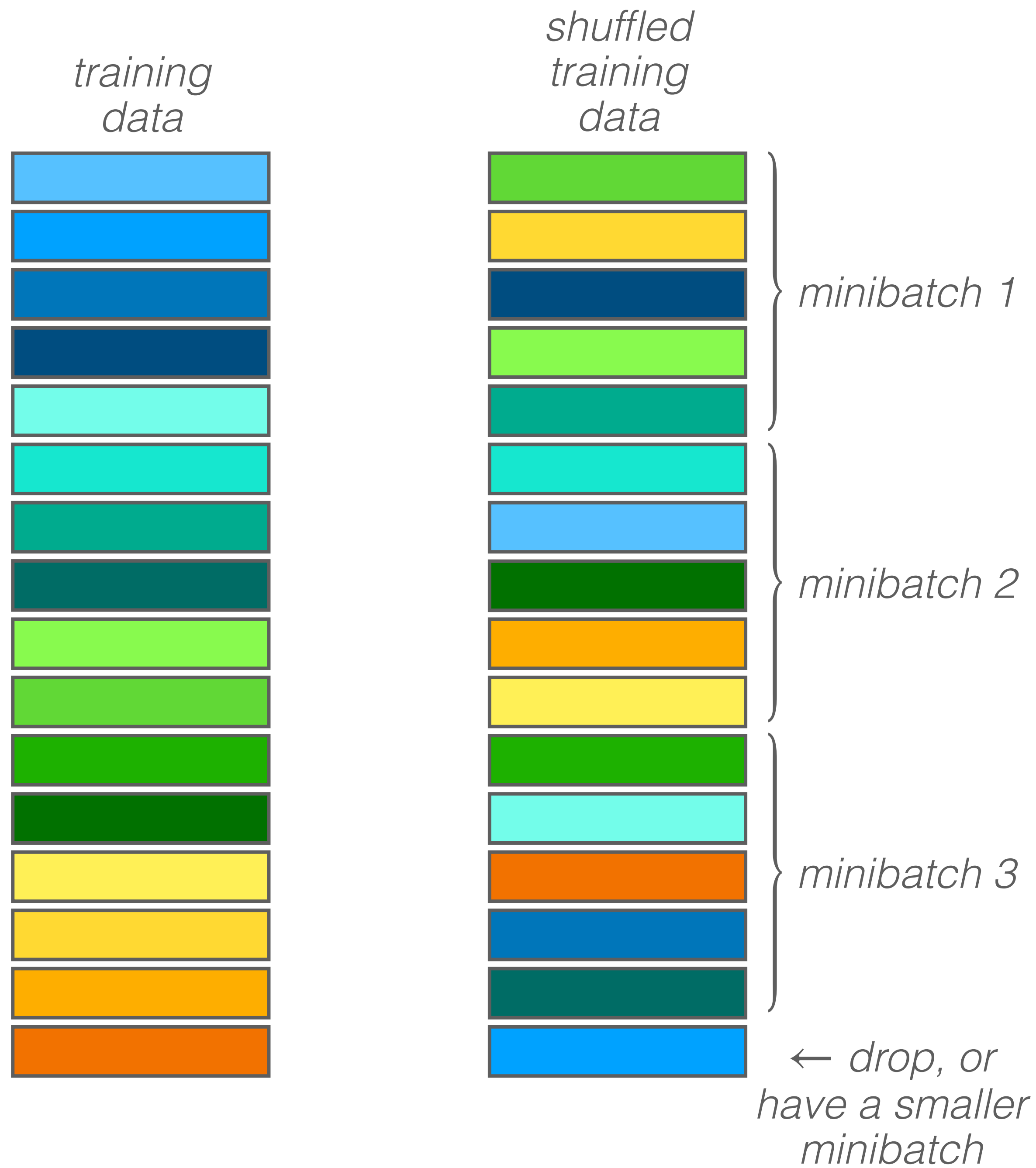
*training
data*



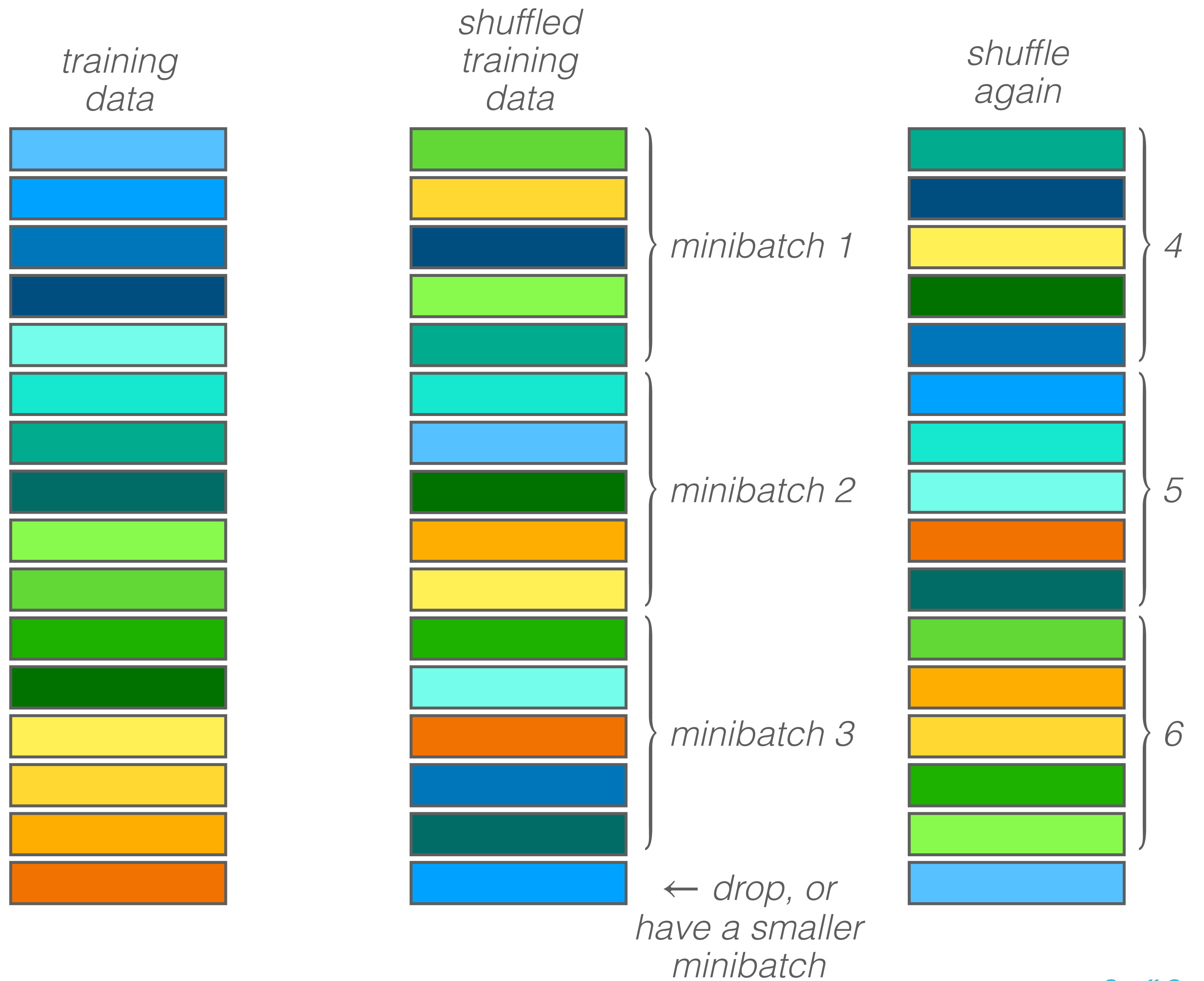
*shuffled
training
data*



Epochs



Epochs



Gradient descent variants



- $B = N$ gradient descent
- $B \ll N$ minibatch SGD
- $B = 1$ SGD

Smaller B takes more iterations, but
each iteration is cheaper

Scaling batch size and learning rate

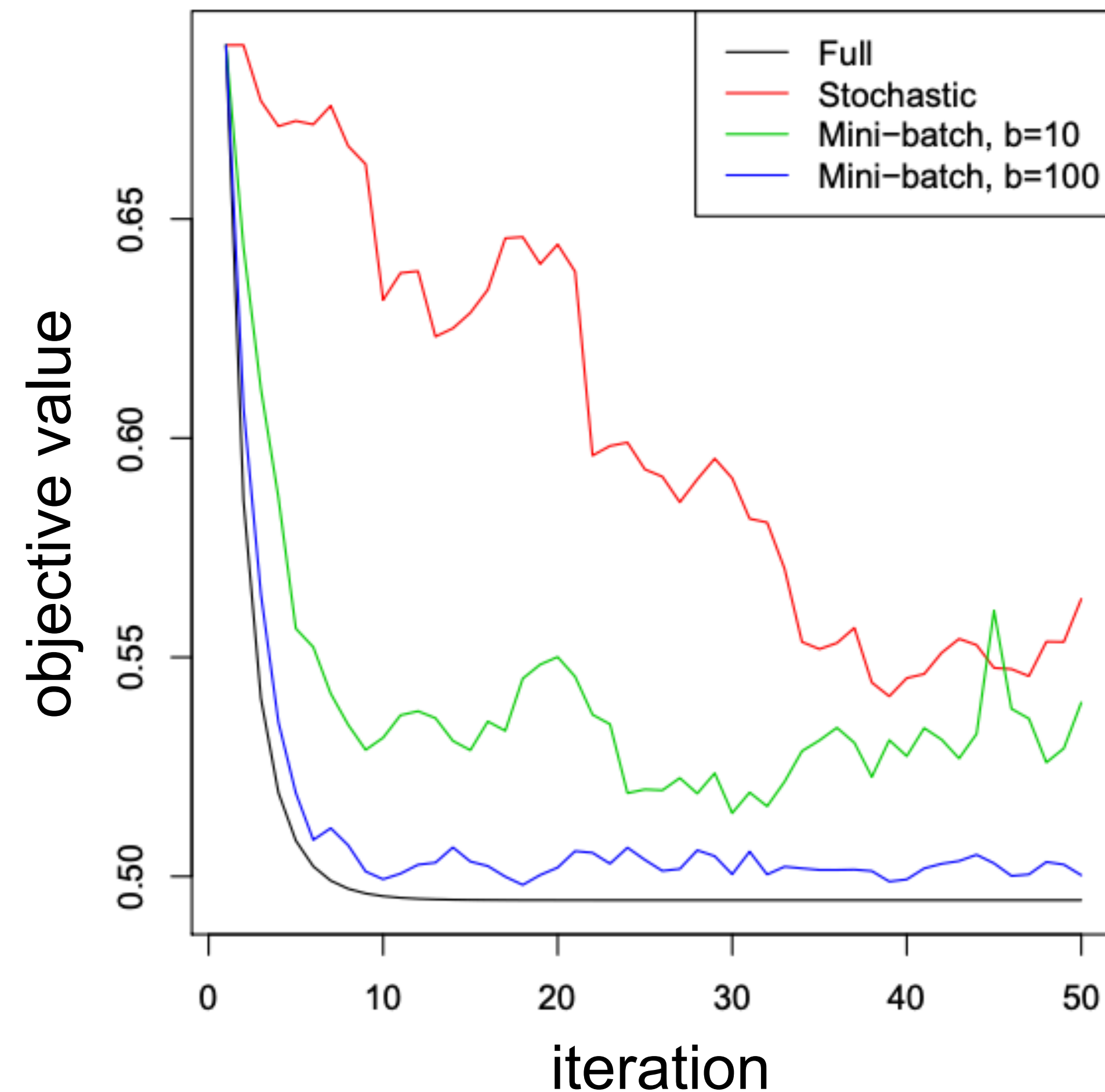
- For small learning rate and batch size, scaling $\eta \propto B$ leaves trajectory approximately unchanged
 - ▶ step direction becomes identical when $\eta \rightarrow 0$
- This scaling also leaves compute unchanged: two steps at η, B cost the same as one step at $2\eta, 2B$
- As we increase η we start to become curvature-limited
 - ▶ best η (for 1-step decrease in L) goes up slower than B
 - ▶ total work for a given improvement in L goes up
 - ▶ but bigger B means better parallelization, so wall-clock time can still go down
- Around *critical batch size*, scaling is $\eta \propto \sqrt{B}$
 - ▶ CBS: noise and deterministic curvature match effect sizes

SGD vs. Mb-SGD vs. GD

Compare **number of iterations**
vs. **objective value**

- This ex: logistic regression with $n = 10,000, k = 20$
- Fixed step size

→ *Larger batches need fewer iterations, make convergence more stable*

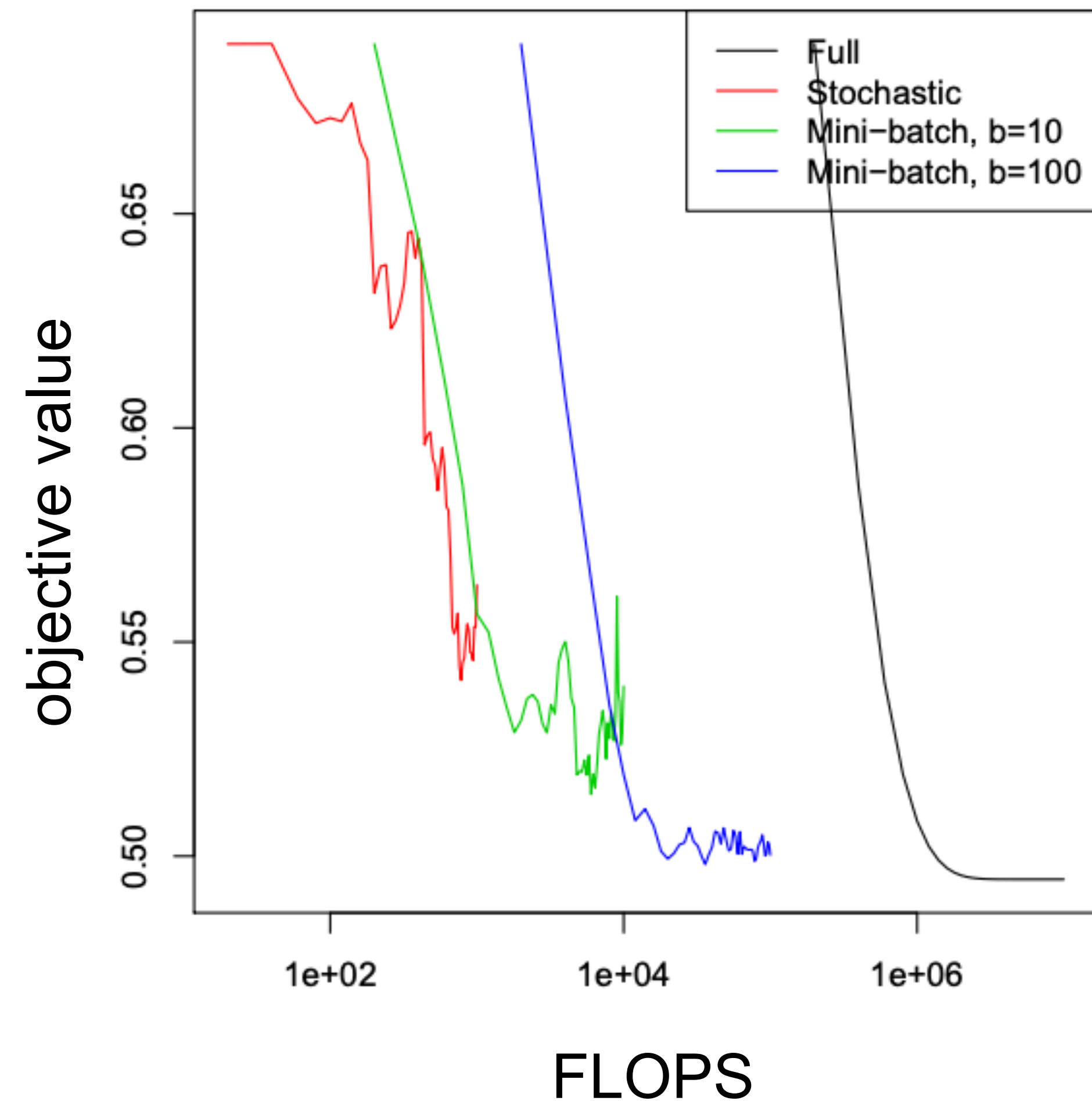


SGD vs. Mb-SGD vs. GD

Compare **FLOPs** vs. **objective value**

- FLOPs = floating point operations (compute cost)

→ *Although Mb-SGD converges more slowly than GD terms of iterations, it may converge more quickly in terms of FLOPs*



note log scale

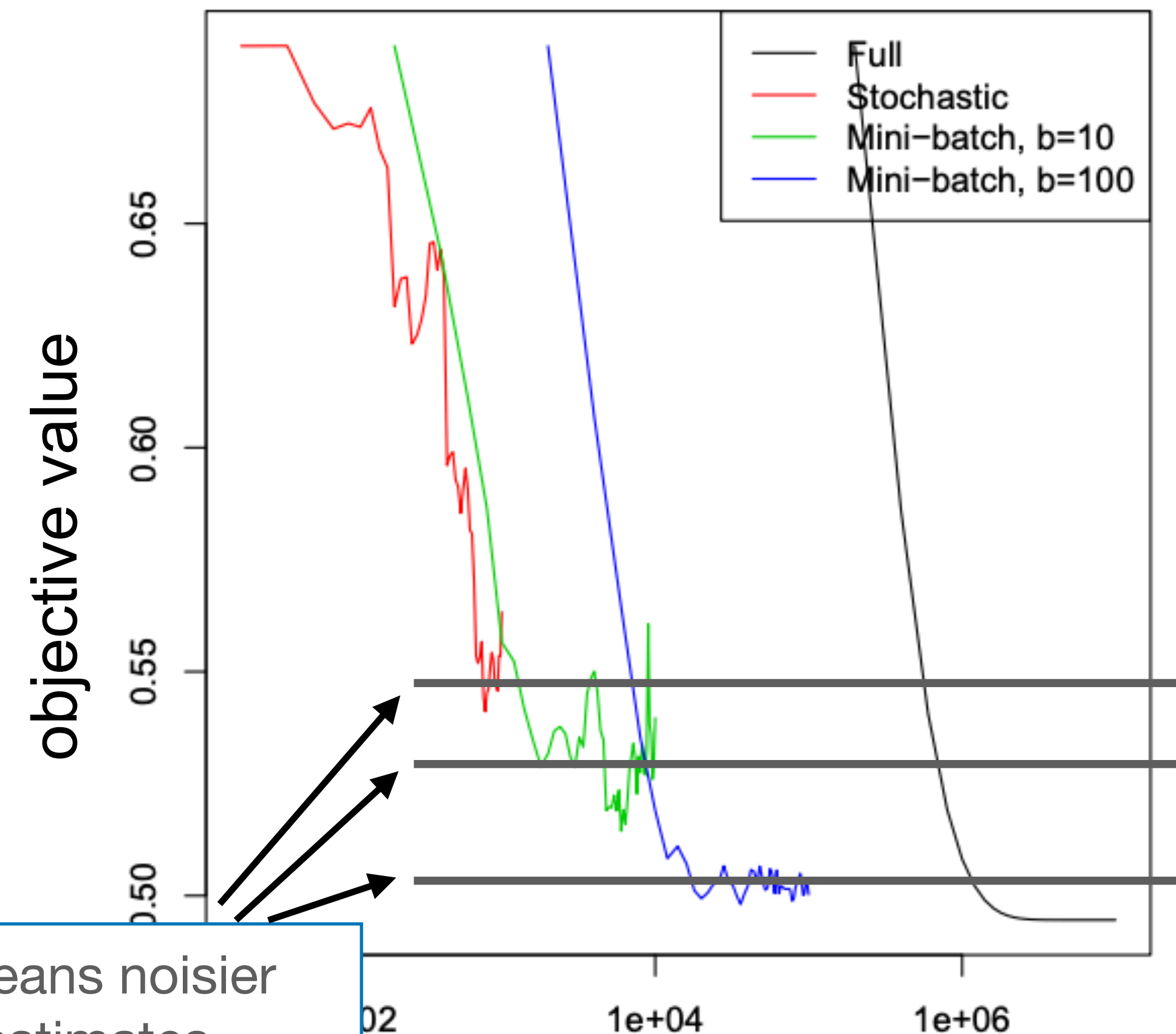
Credit: Ryan
Tibshirani

SGD vs. Mb-SGD vs. GD

Compare **FLOPs** vs. **objective value**

- FLOPs = floating point operations (compute cost)

→ *Although Mb-SGD converges more slowly than GD terms of iterations, it may converge more quickly in terms of FLOPs*



Smaller b means noisier gradient estimates
We are keeping constant η so we reach the random-walk stage at higher error

FLOPS

note log scale

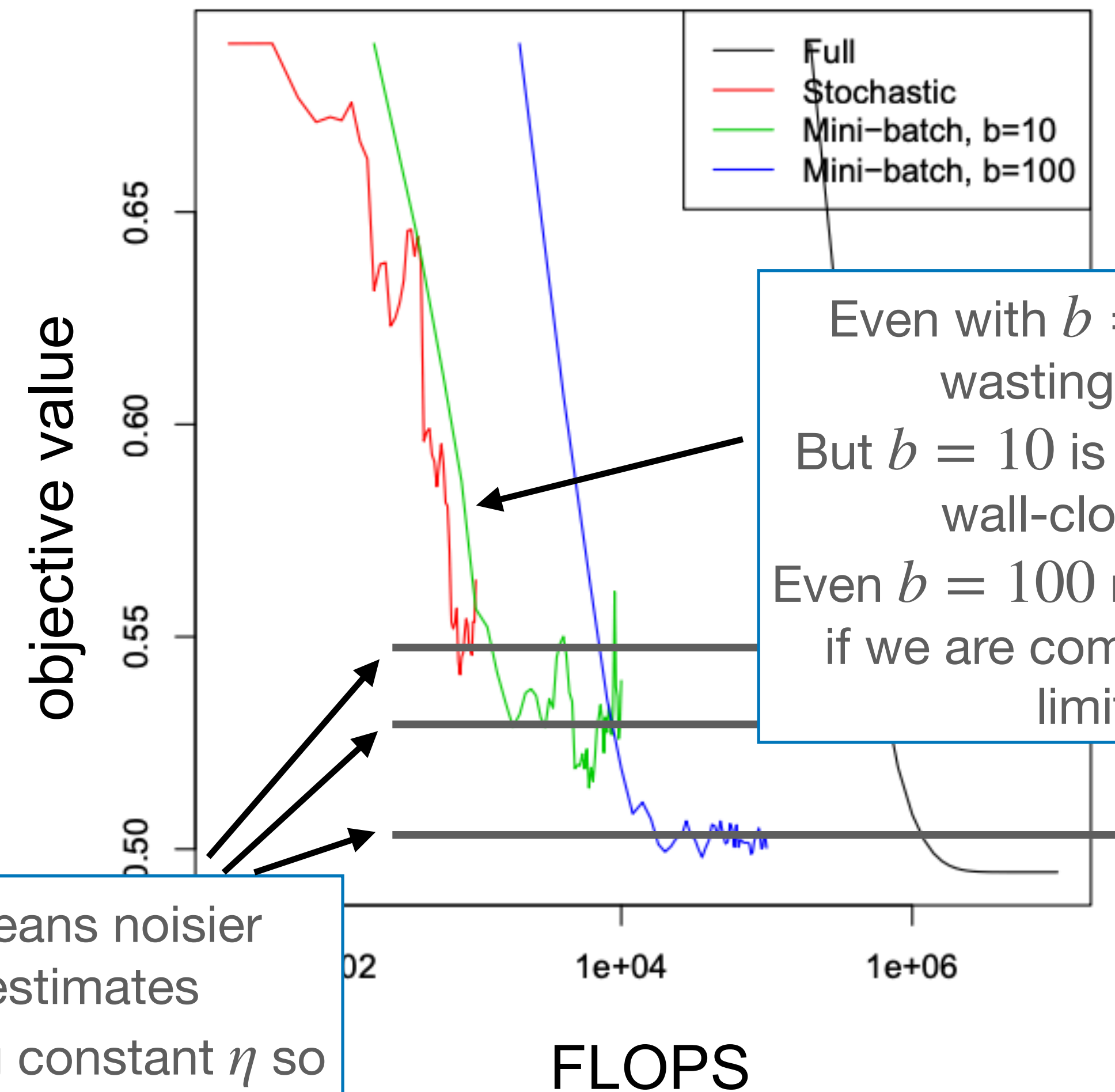
Credit: Ryan
Tibshirani

SGD vs. Mb-SGD vs. GD

Compare **FLOPs** vs. **objective value**

- FLOPs = floating point operations (compute cost)

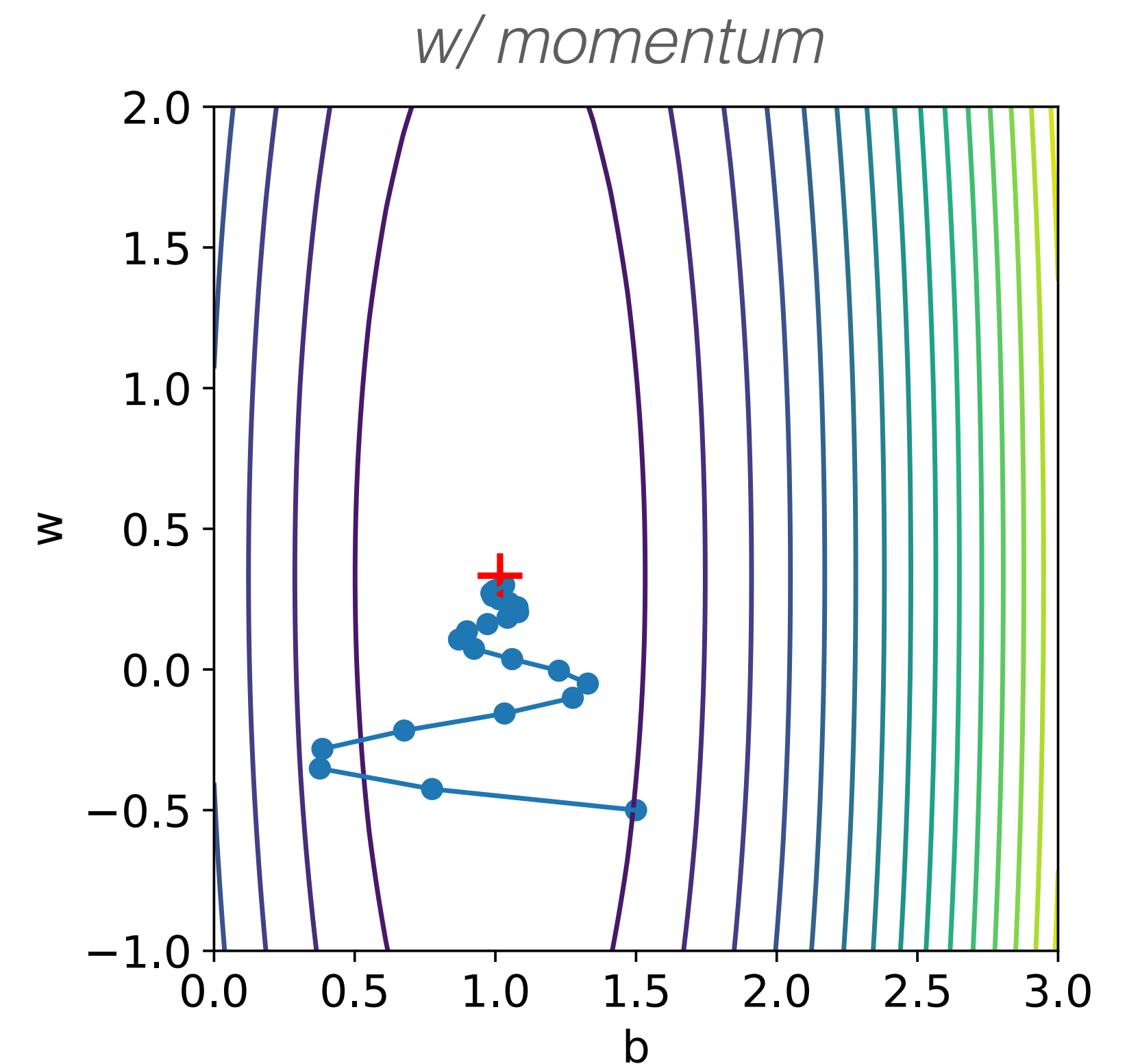
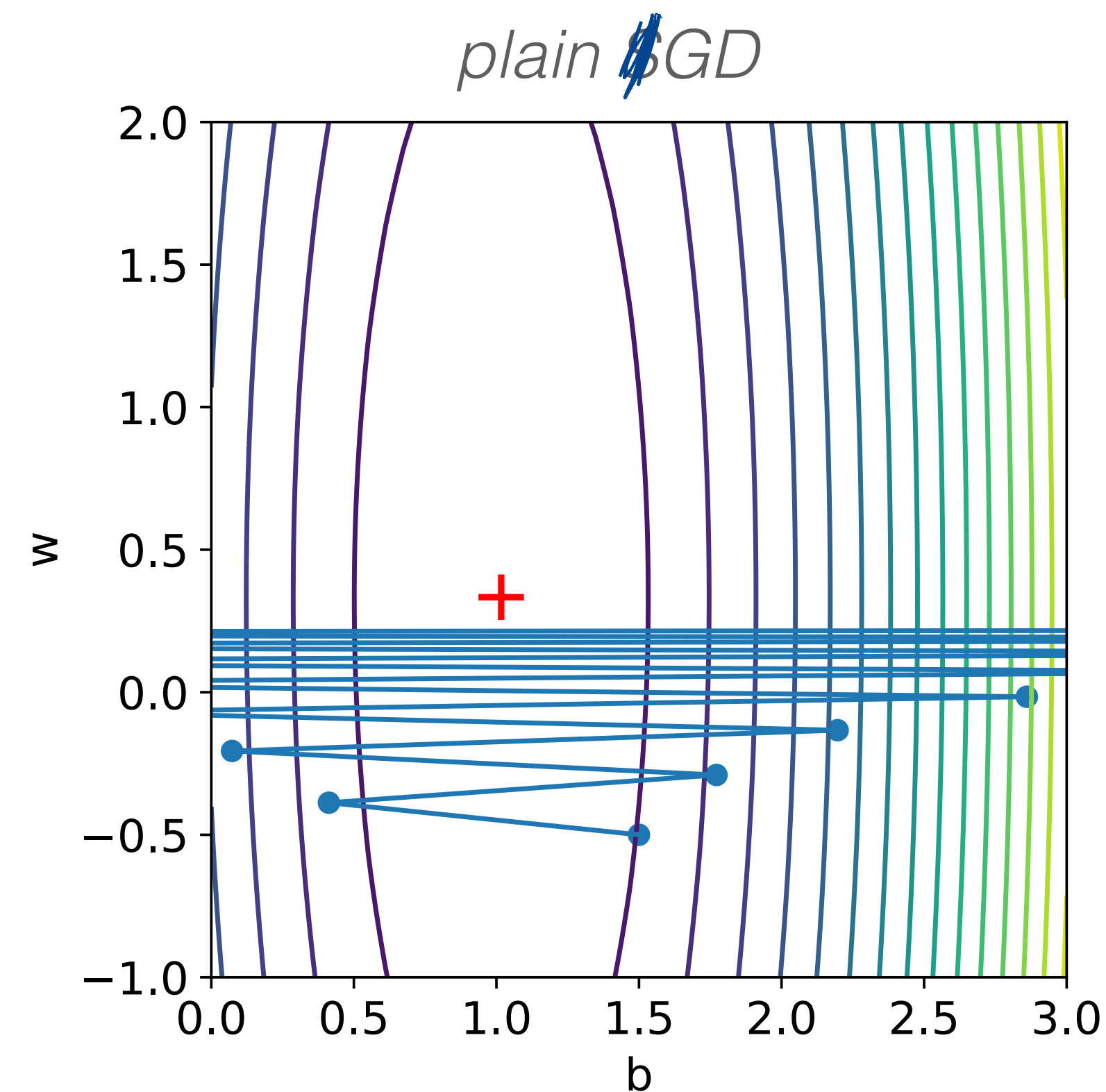
→ *Although Mb-SGD converges more slowly than GD terms of iterations, it may converge more quickly in terms of FLOPs*



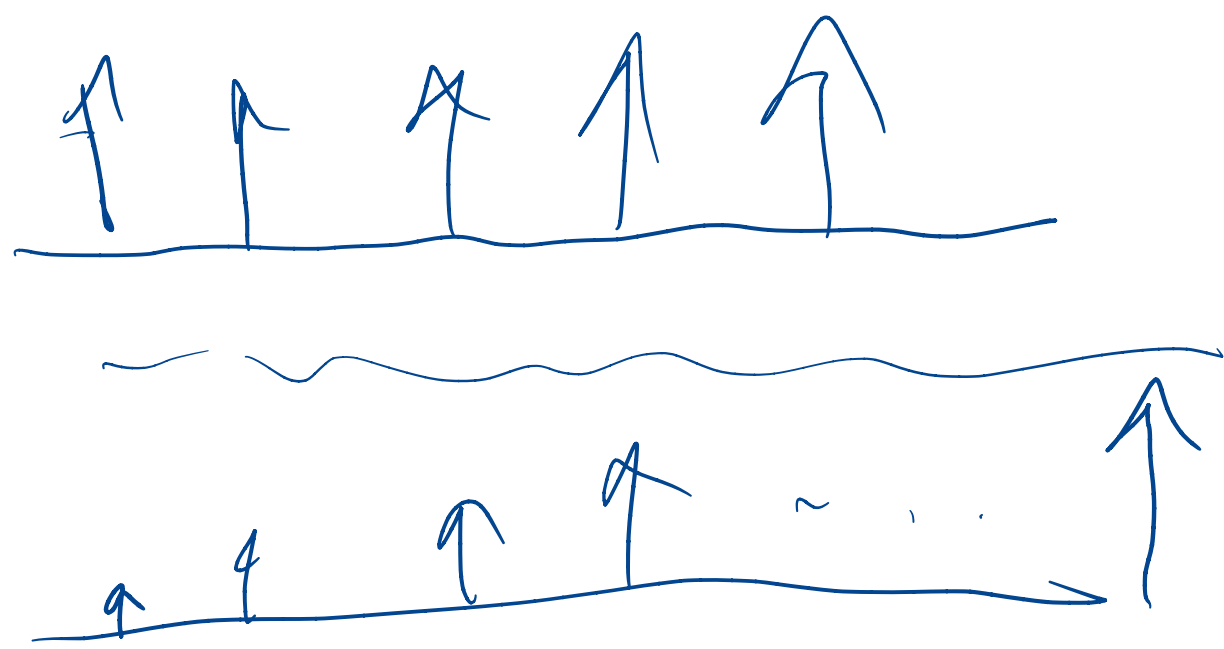
Even with $b = 10$ we are wasting FLOPs
But $b = 10$ is likely faster in wall-clock time
Even $b = 100$ might be faster if we are communication-limited

Smaller b means noisier gradient estimates
We are keeping constant η so we reach the random-walk stage at higher error

Gradient variance



- Momentum helps a bit w/ poor scaling / conditioning
- Can we do better?
- Idea: gradient has high variance in directions where we're oscillating
- Suggests tracking variance → Adam



Track the gradient variance: Adam

Learning rate scales differently than SGD

- Adam (“ADaptive Moment estimation”):

$$m_{t+1} = \beta_m m_t + (1 - \beta_m) g_t$$

gradient estimate g_t

$$v_{t+1} = \beta_v v_t + (1 - \beta_v) g_t^2$$

$(\cdot)^2$ is componentwise

$$\hat{m}_{t+1} = \frac{1}{1 - \beta_m^t} m_{t+1}, \quad \hat{v}_{t+1} = \frac{1}{1 - \beta_v^t} v_{t+1}$$

transient correction

$$\theta_{t+1} = \theta_t - \eta \frac{1}{\sqrt{\hat{v}_{t+1}} + \epsilon} \hat{m}_{t+1}$$

$\sqrt{\cdot}$ is componentwise

- Four hyperparameters: moving average $\beta_m, \beta_v \in (0, 1)$, learning rate $\eta > 0$, curvature regularizer $\epsilon > 0$ (& $\theta^{(1)}$)
- Often works very well — but may not converge at all! (failure cases as simple as $\min_w \|w\|^2$!)
- there are other methods that try to use gradient variance while guaranteeing better theoretical properties, but none uniformly better than Adam in practice

Learning rate scales differently than SGD

Track the gradient variance: Adam

AdamW: handle weight decay separately — g_t excludes ridge, separately multiply θ_t by a factor < 1

- Adam (“ADaptive Moment estimation”):

$$m_{t+1} = \beta_m m_t + (1 - \beta_m) g_t$$

gradient estimate g_t

$$v_{t+1} = \beta_v v_t + (1 - \beta_v) g_t^2$$

$(\cdot)^2$ is componentwise

$$\hat{m}_{t+1} = \frac{1}{1 - \beta_m^t} m_{t+1}, \quad \hat{v}_{t+1} = \frac{1}{1 - \beta_v^t} v_{t+1}$$

transient correction

$$\theta_{t+1} = \theta_t - \eta \frac{1}{\sqrt{\hat{v}_{t+1}} + \epsilon} \hat{m}_{t+1}$$

$\sqrt{\cdot}$ is componentwise

- Four hyperparameters: moving average $\beta_m, \beta_v \in (0, 1)$, learning rate $\eta > 0$, curvature regularizer $\epsilon > 0$ (& $\theta^{(1)}$)
- Often works very well — but may not converge at all! (failure cases as simple as $\min_w \|w\|^2$!)
 - there are other methods that try to use gradient variance while guaranteeing better theoretical properties, but none uniformly better than Adam in practice

Learning objectives— convexity, gradient descent

- When would we choose gradient descent vs. exact for linear regression? How would we implement either (w/ access to NumPy and Torch)?
- What is a convex function? A convex set?
- How does convexity affect the set of local or global optima? How does it affect gradient descent?
- When is the optimal solution for linear regression well-determined, poorly-determined, or undetermined?
- What effect does a poorly-determined solution have on gradient descent? How can we mitigate this?
- How can we use (minibatch) SGD to scale to large datasets and take advantage of parallel compute?

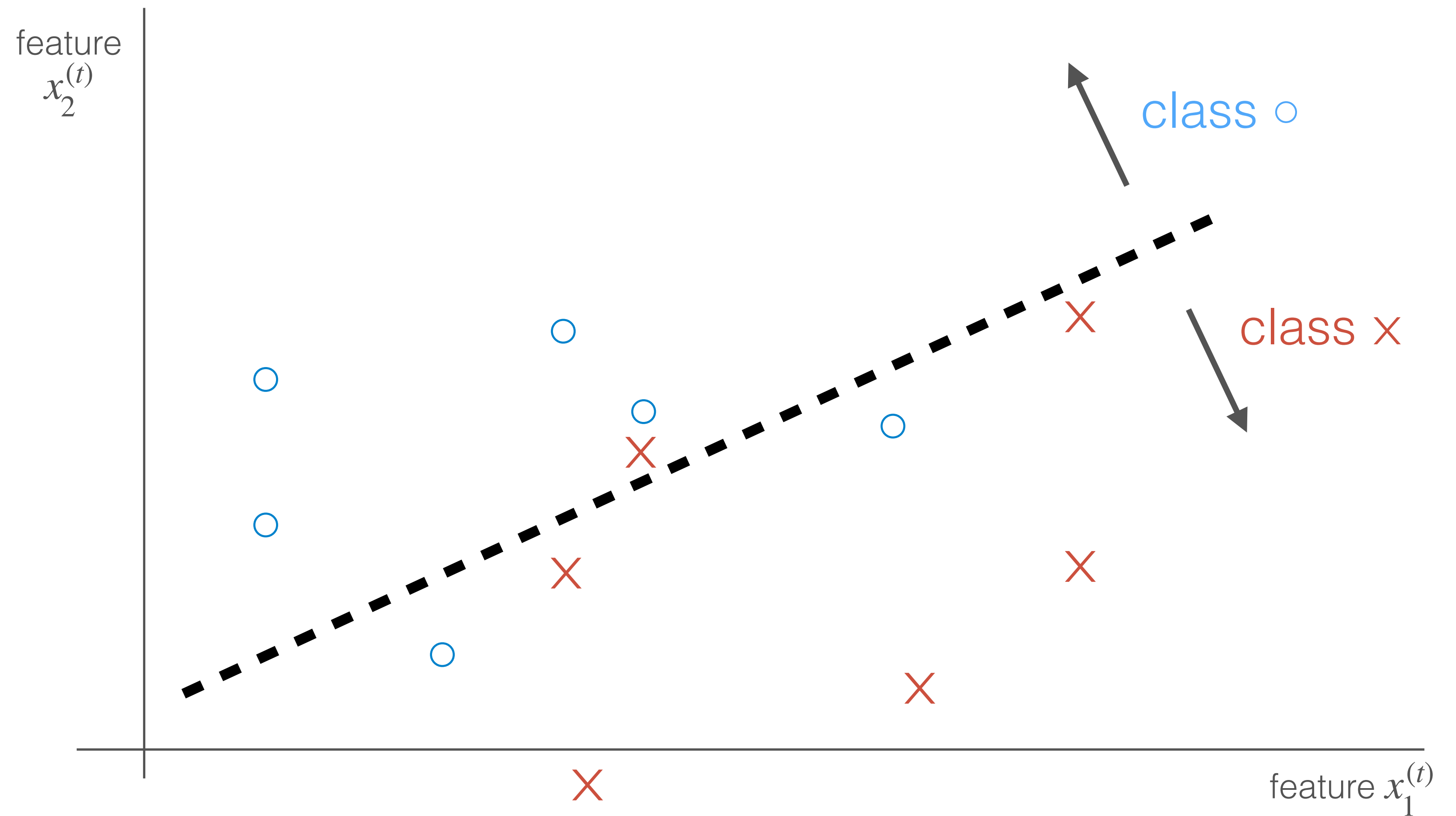
Linear classifiers



10-701 Introduction to Machine Learning
Geoff Gordon

Linear classifier

goal: intuition about
geometry of vector
spaces, linear functions,
linear classifiers



cf. last layer of a neural network

In higher dimensions

*boundary can be a
point, line, plane,
hyperplane —
always 1d lower than
feature space*

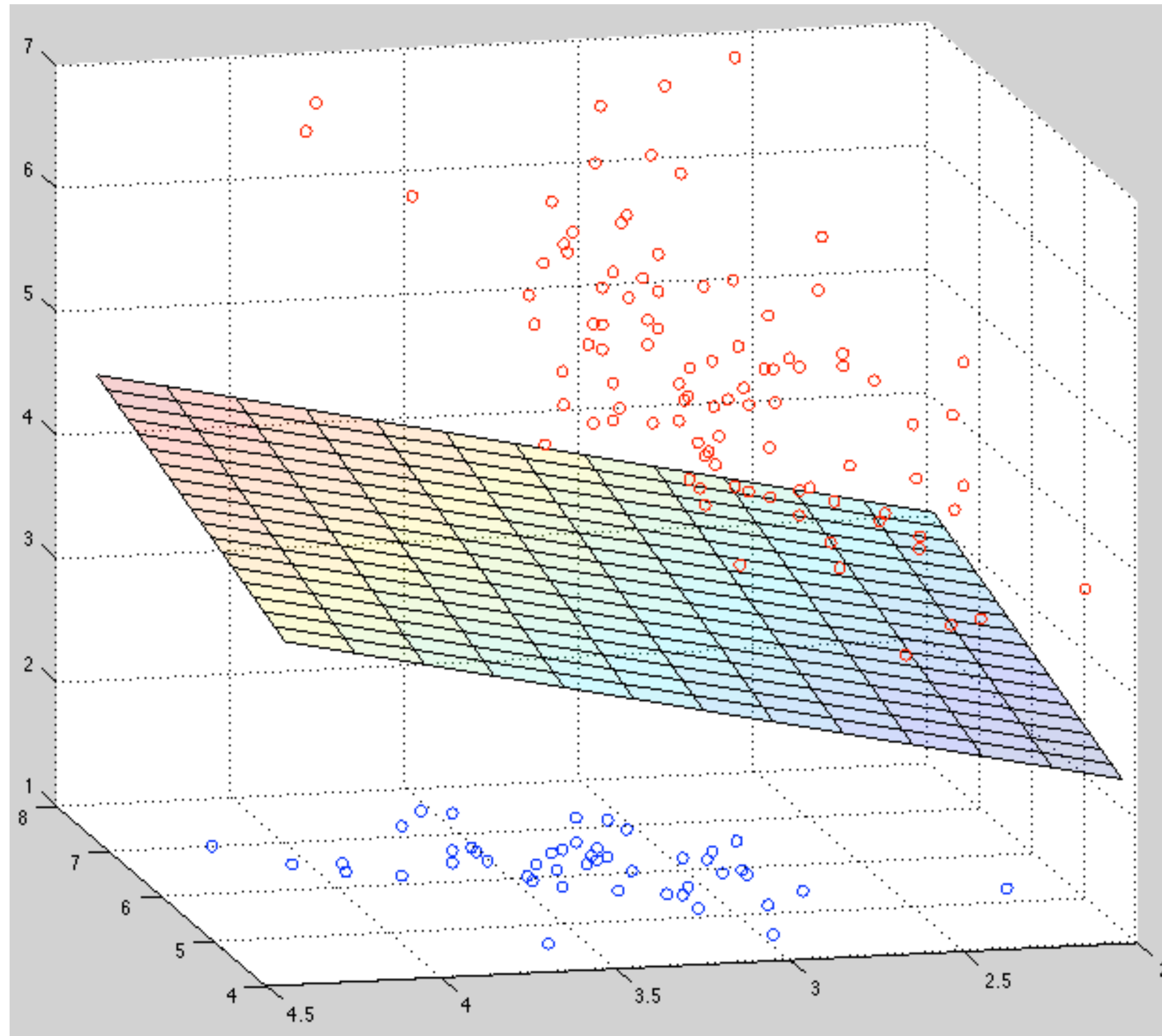


Figure courtesy of Matt Gormley

Linear functions and linear classifiers

Today, classes are +1 and -1 (can use any two labels)

a linear function
(the discriminant)

- Predict $\text{sgn}(\overbrace{w \cdot x^{(t)} + b})$

$$\text{sgn}(z) = \begin{cases} +1 & z > 0 \\ -1 & z < 0 \\ 0 & z = 0 \end{cases}$$

← positive class

← negative class

← ??? (break tie)

- Linear function = represent w/ inner product:

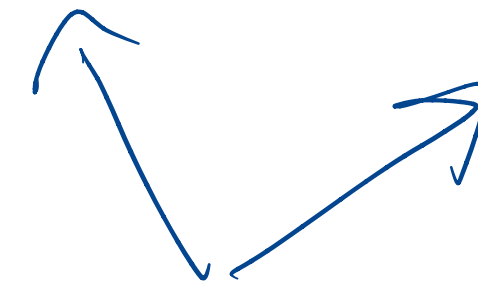
$$w \cdot x = w^T x = \sum_i w_i x_i$$

e.g., $w = \begin{pmatrix} 3 \\ 2 \end{pmatrix}$ $x = \begin{pmatrix} 1 \\ -1 \end{pmatrix}$ $w \cdot x = 3 - 2 = 1$

we won't distinguish linear vs. affine
(equally expressive for our purposes
due to constant feature, but sometimes
convenient to break out the bias term)

Inner product lets us define

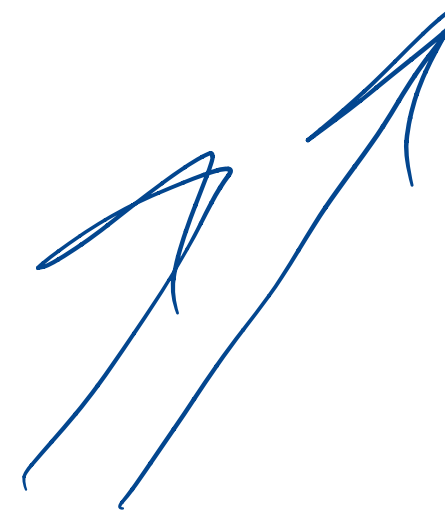
- Orthogonal / normal



$$w \cdot x = 0$$

$$w \perp x$$

- Parallel



$$w = \lambda x$$

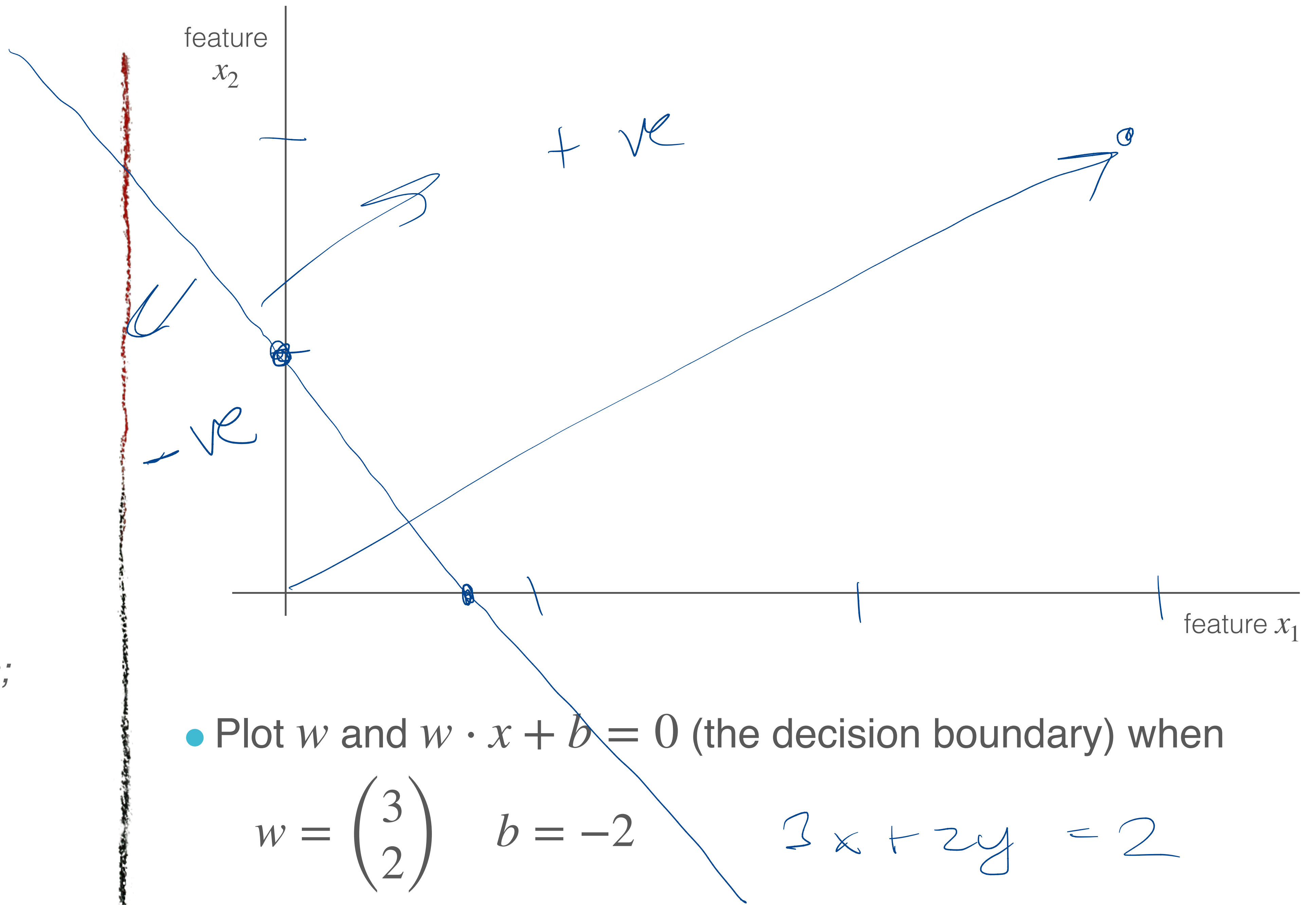
$$w \parallel x$$

- Length / norm

$$\|w\| = \sqrt{w \cdot w}$$

Decision boundary example

note intercepts;
normal vector

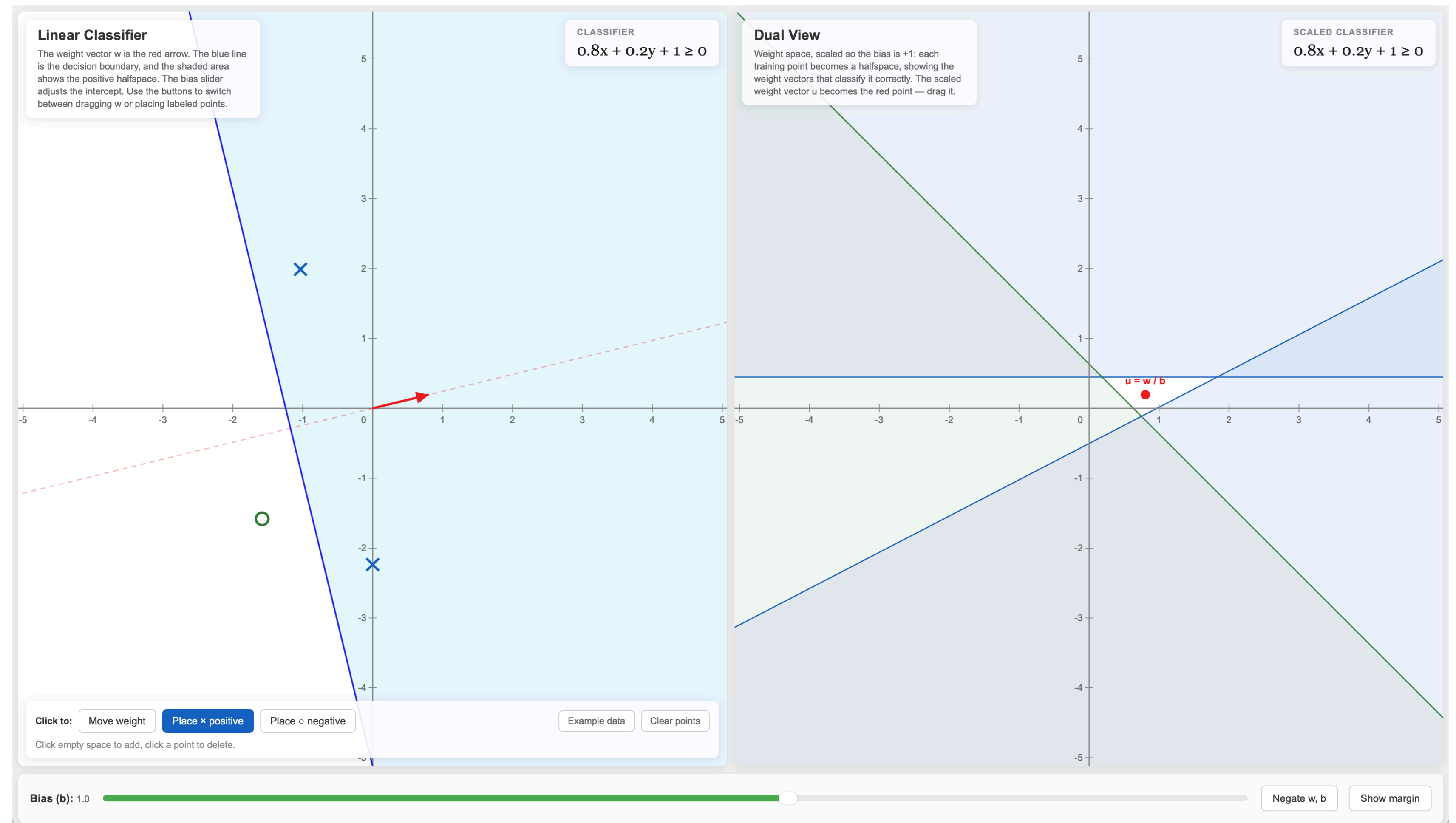


Examples as constraints

- Each example (x, y) gives us a constraint on w
 $w \cdot x + b > 0$ for positive ex ($y = +1$)
 $w \cdot x + b < 0$ for negative ex ($y = -1$)
- Numerical value of discriminant tells us whether the constraint is satisfied or violated
 - ▶ $|w \cdot x + b|$ is called the **slack** if constraint is satisfied, and the **violation** if not
- Notation hack: can combine both constraints as
 $y(w \cdot x + b) > 0$
 - ▶ flip all negative examples across origin (incl. constant feature) and make them positive

Linear classifier geometry

interactive demo



- Decision boundary and normal vector
- Effect of changing b ; scaling w, b ; sign flip
- Dual view and version space

Perceptron

- Oldest method for learning linear classifiers is ***perceptron*** (1960, with predecessors in '50s)
- Defined in the *online* or *streaming* learning model
- Online model: repeat
 - ▶ find out $X^{(t)}$
 - ▶ predict $\hat{Y}^{(t)}$
 - ▶ find out $Y^{(t)}$, pay $\ell(\hat{Y}^{(t)}, Y^{(t)})$
 - ▶ $t++$

Online vs. batch

- Cf. *batch* model from earlier lectures:
 - ▶ find out all of $X^{(t)}, Y^{(t)}$ for $t = 1 : N$
 - ▶ predict $\hat{Y}^{(t)}$ for separate test set inputs $X^{(t)}$, $t = N + 1 : N + N'$
 - ▶ find out $Y^{(t)}$ on test set, pay $\sum_{t=N+1}^{N+N'} \ell(\hat{Y}^{(t)}, Y^{(t)})$
- Can often convert methods for learning in one model to work in the other model
 - ▶ e.g., ***follow-the-leader***: when learning online, run a batch learner on examples so far, use it to predict next
 - ▶ or ***online-to-batch***: run an online learner on the batch training set, take random / best / voted / last hypothesis

Perceptron

- Always predict $\hat{Y}^{(t)} = \text{sgn}(w \cdot X^{(t)})$
 - ▶ or $\hat{Y}^{(t)} = \text{sgn}(w \cdot X^{(t)} + b)$ if separate intercept
- Each time we make a mistake, $w \leftarrow w + Y^{(t)}X^{(t)}$
 - ▶ and $b \leftarrow b + Y^{(t)}$ if separate intercept

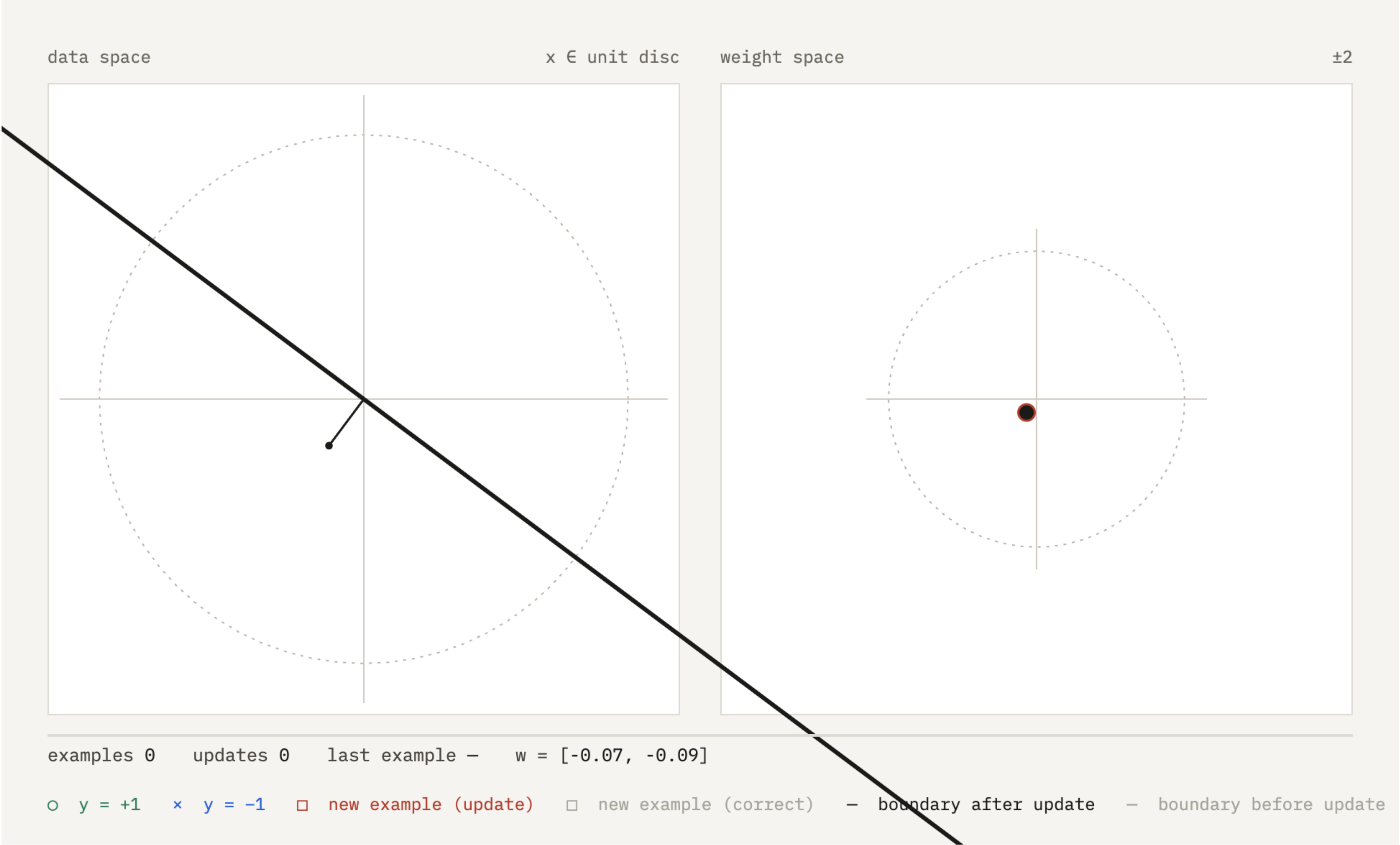
- Why might this work?

- ▶ $(w + Y^{(t)}X^{(t)}) \cdot X^{(t)} = w \cdot X^{(t)} + \underbrace{Y^{(t)}X^{(t)} \cdot X^{(t)}}_{\geq 0}$

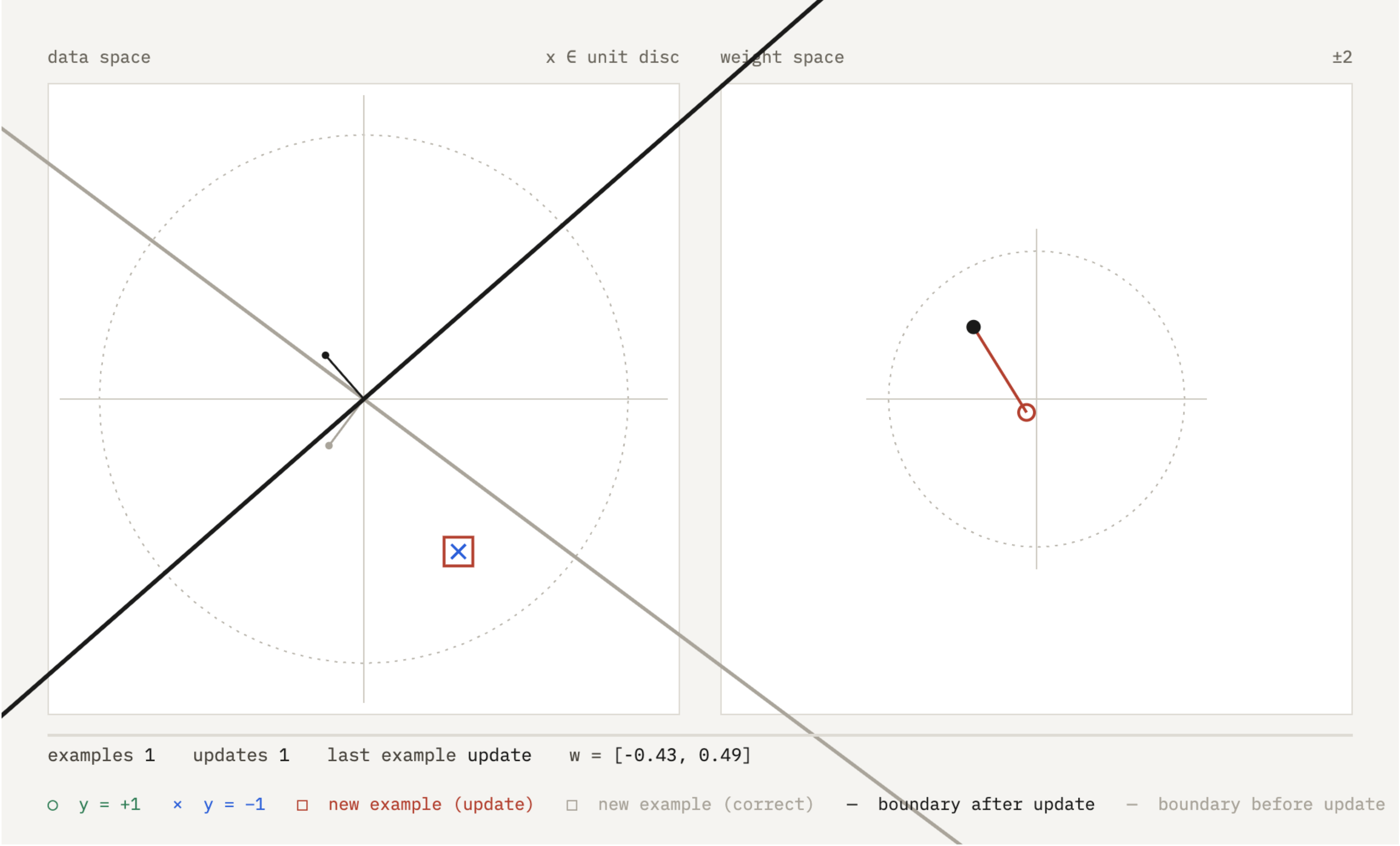
- ▶ compare to $w \cdot X^{(t)}$:

- ▶ if $Y^{(t)} = +1$: *increase*
- ▶ if $Y^{(t)} = -1$: *decrease*

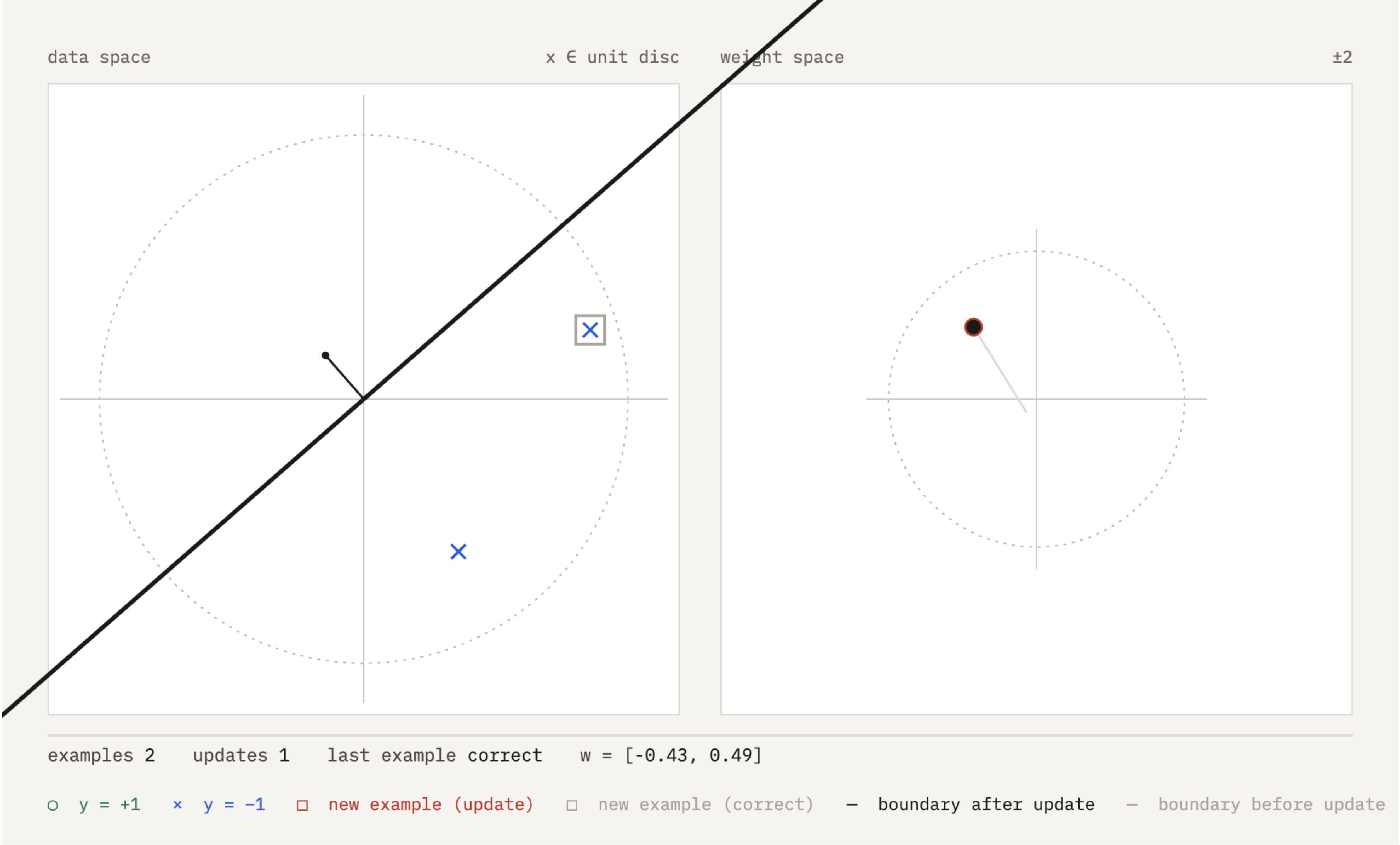
Perceptron



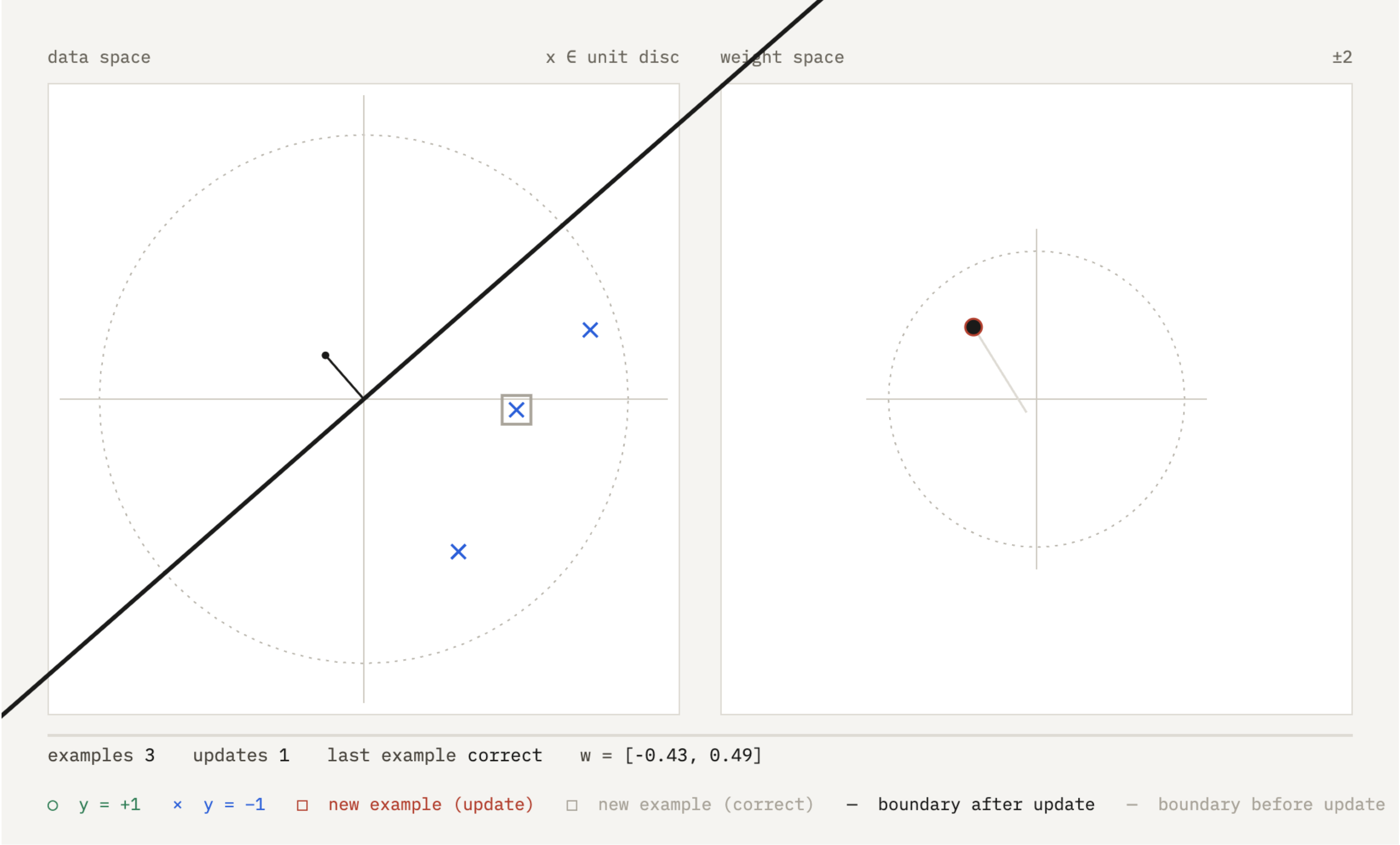
Perceptron



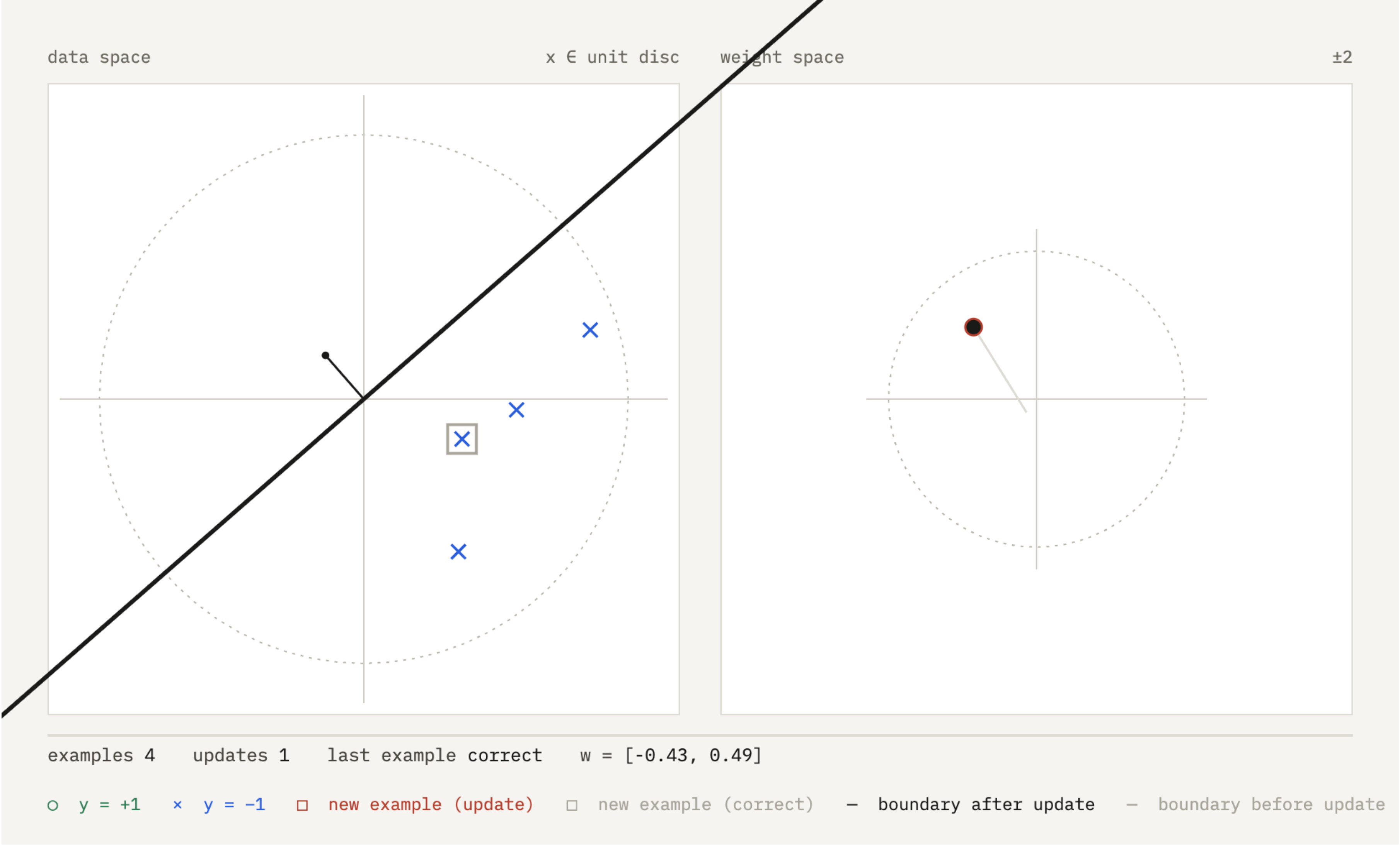
Perceptron



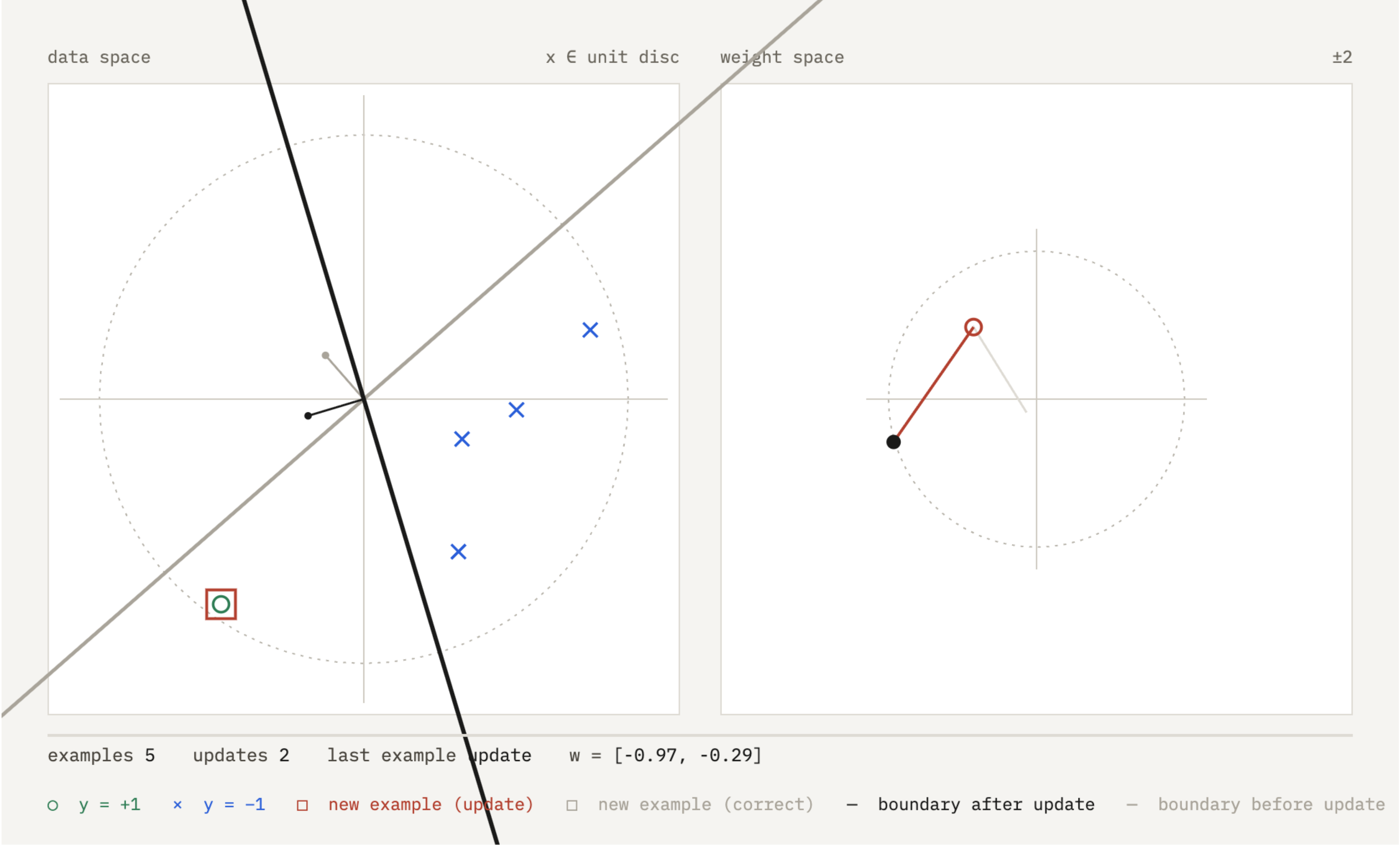
Perceptron



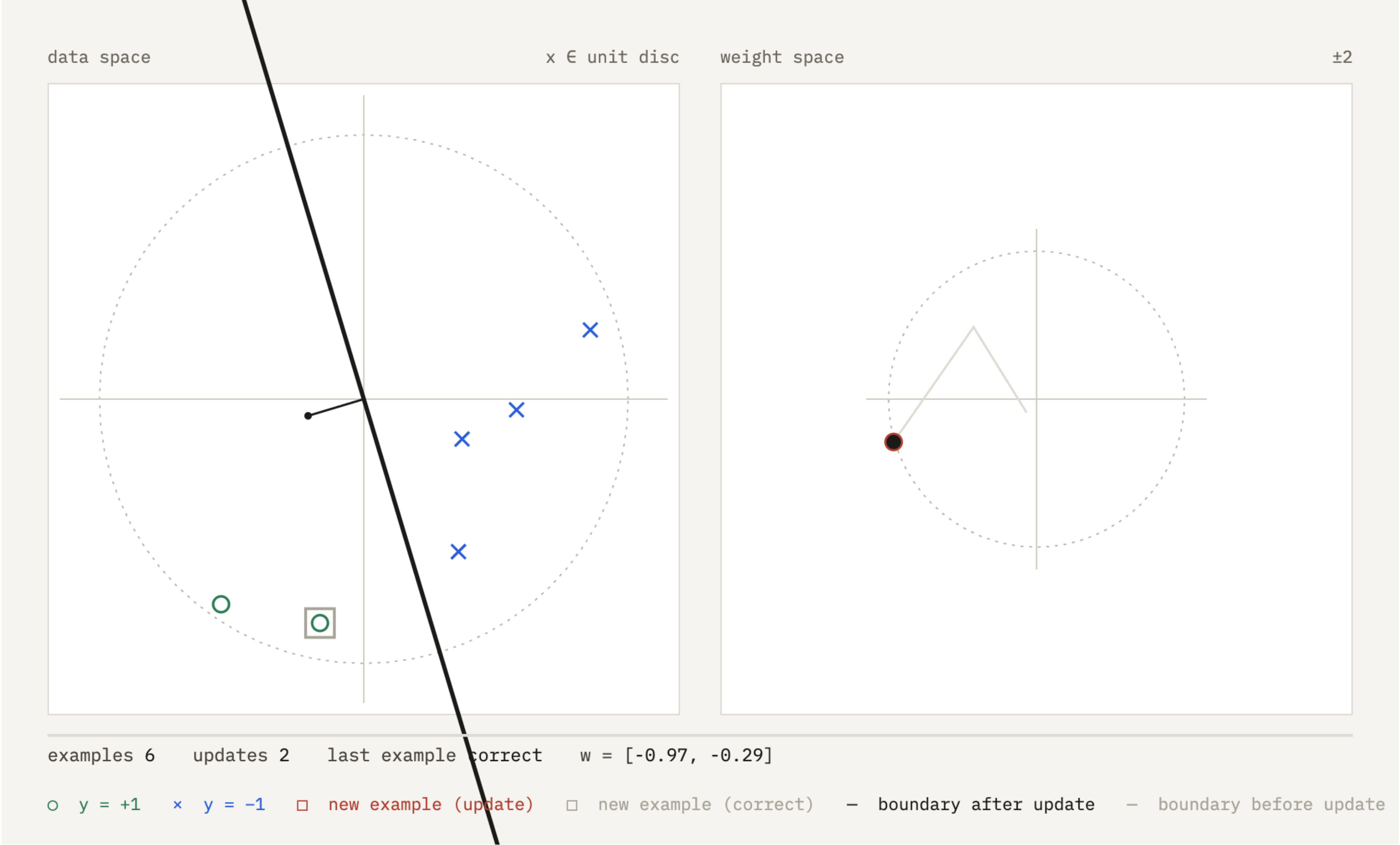
Perceptron



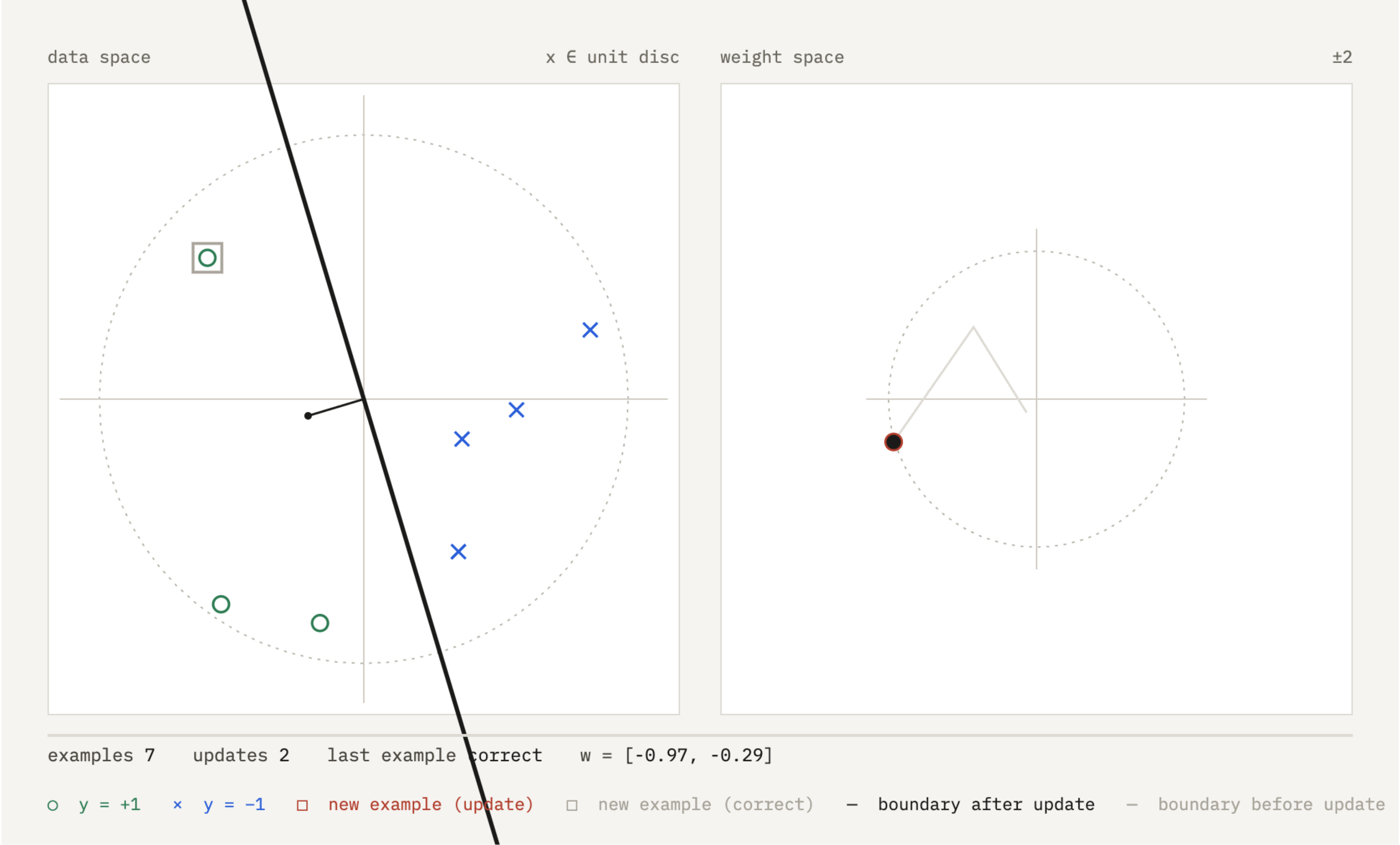
Perceptron



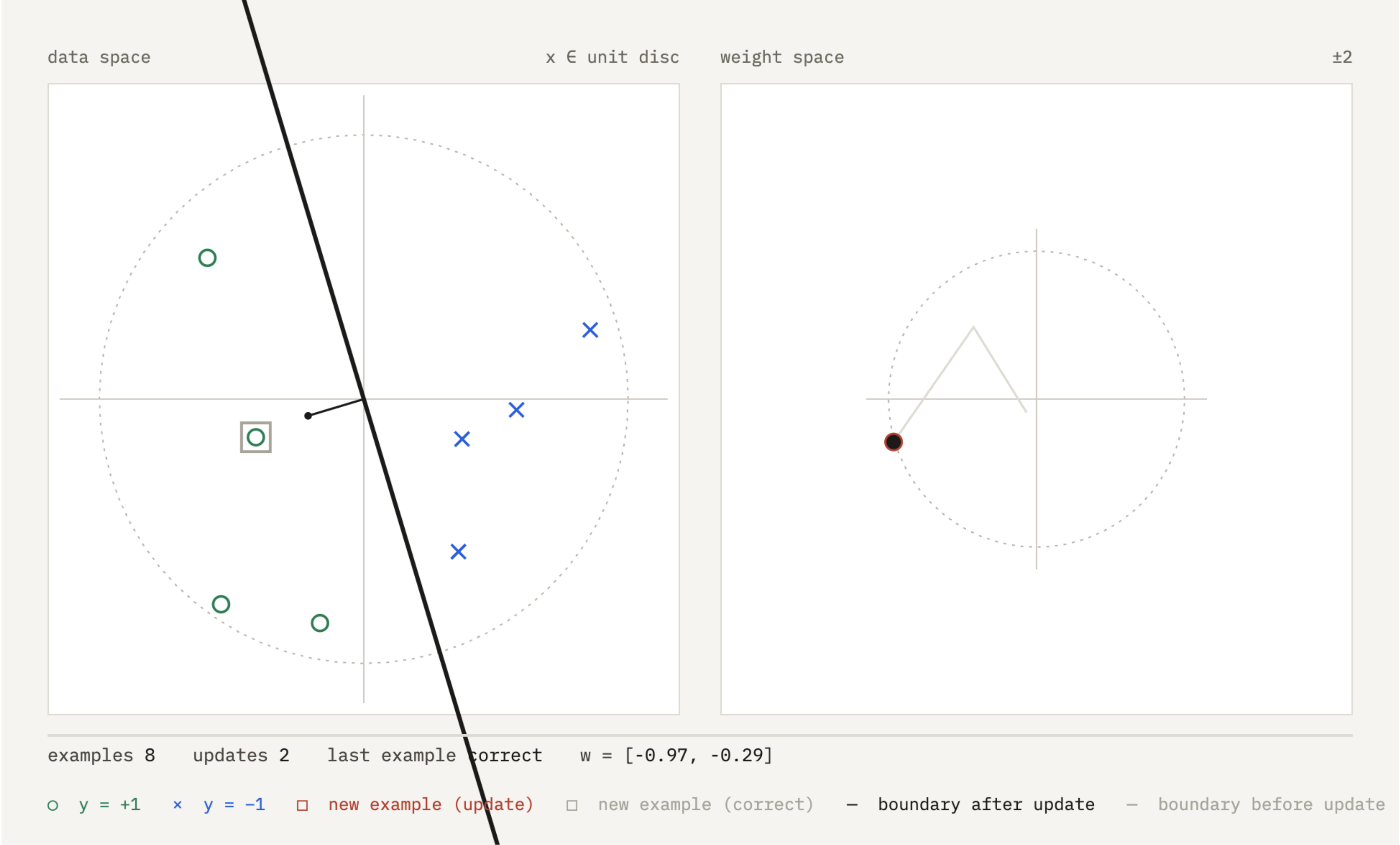
Perceptron



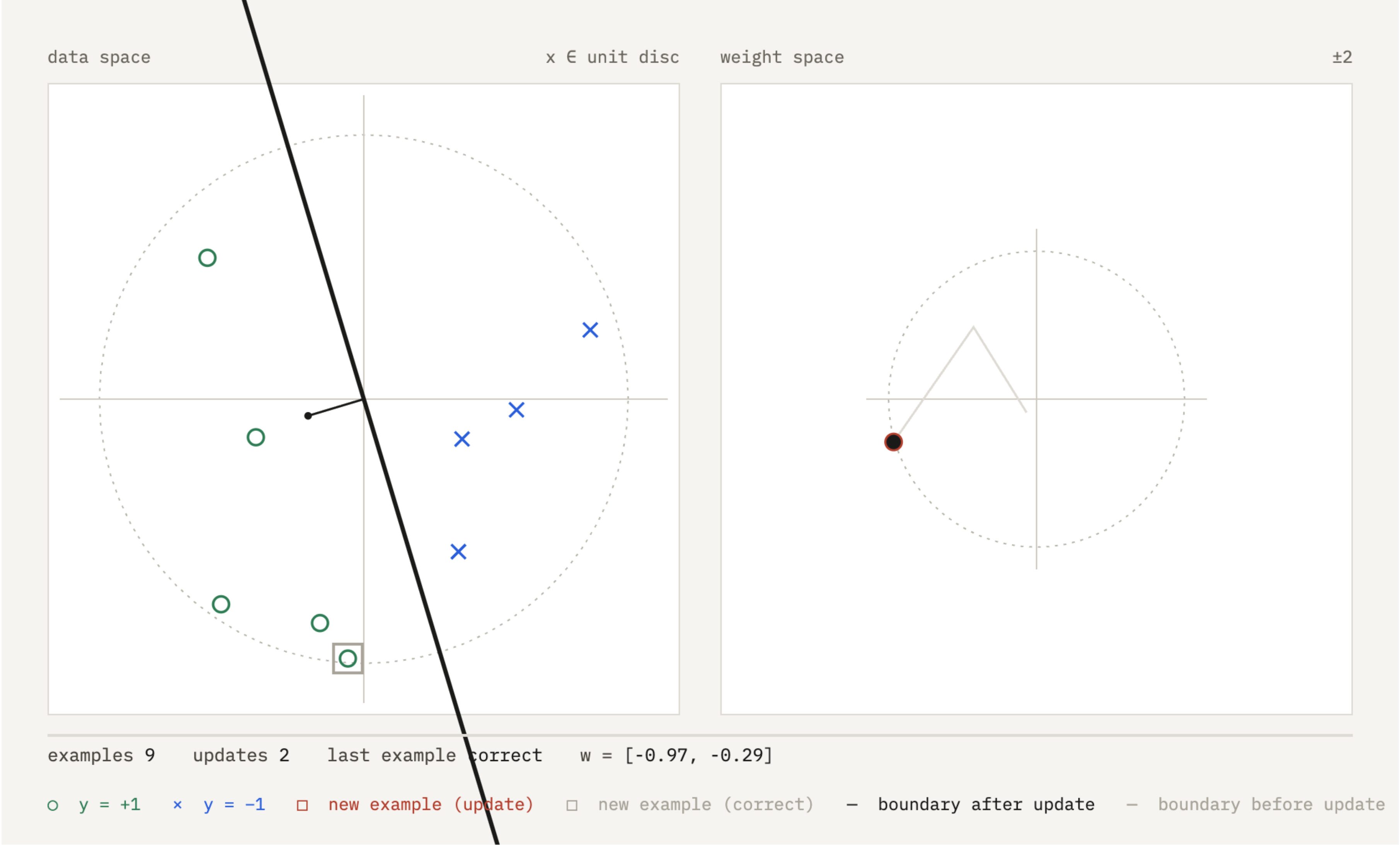
Perceptron



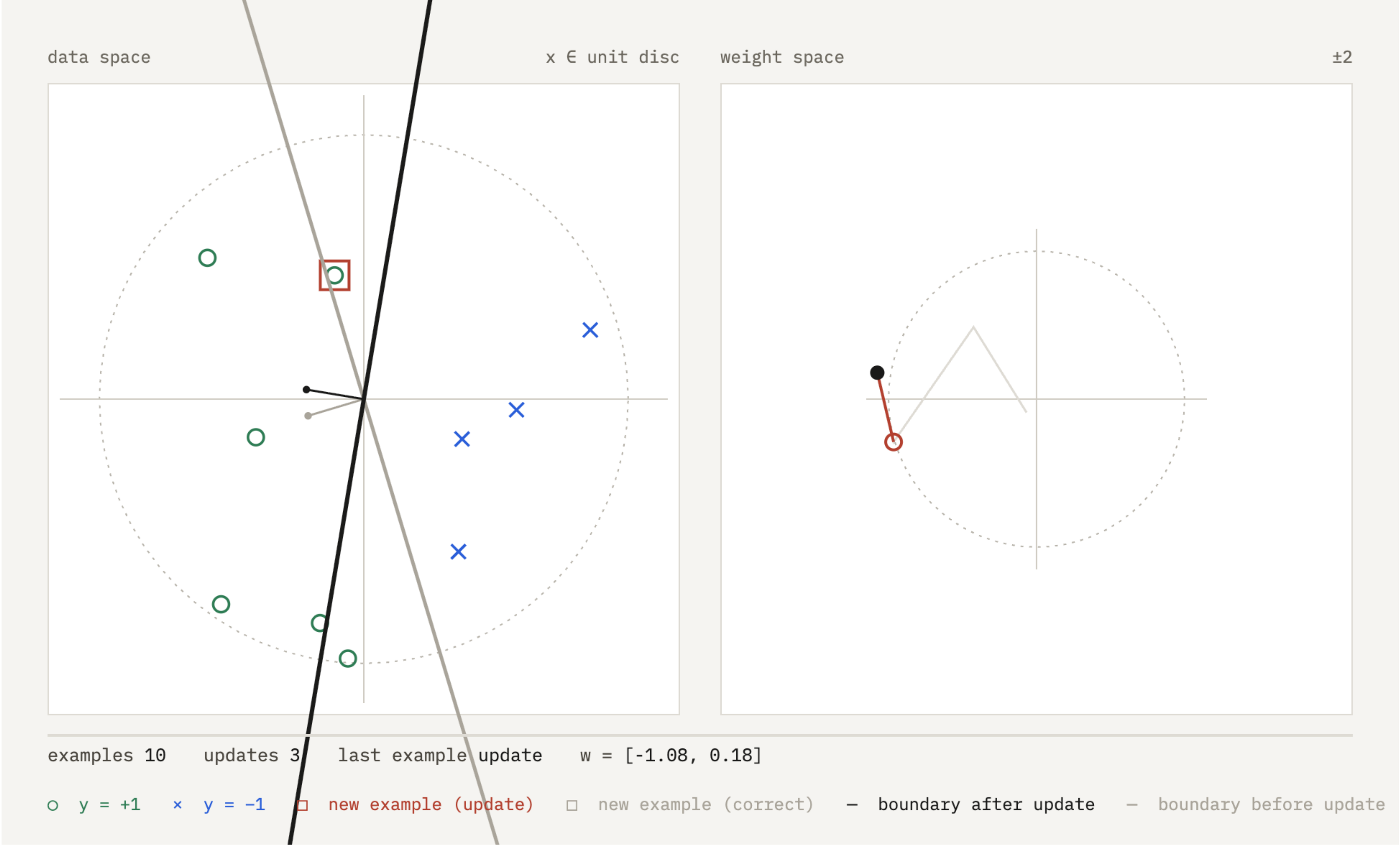
Perceptron



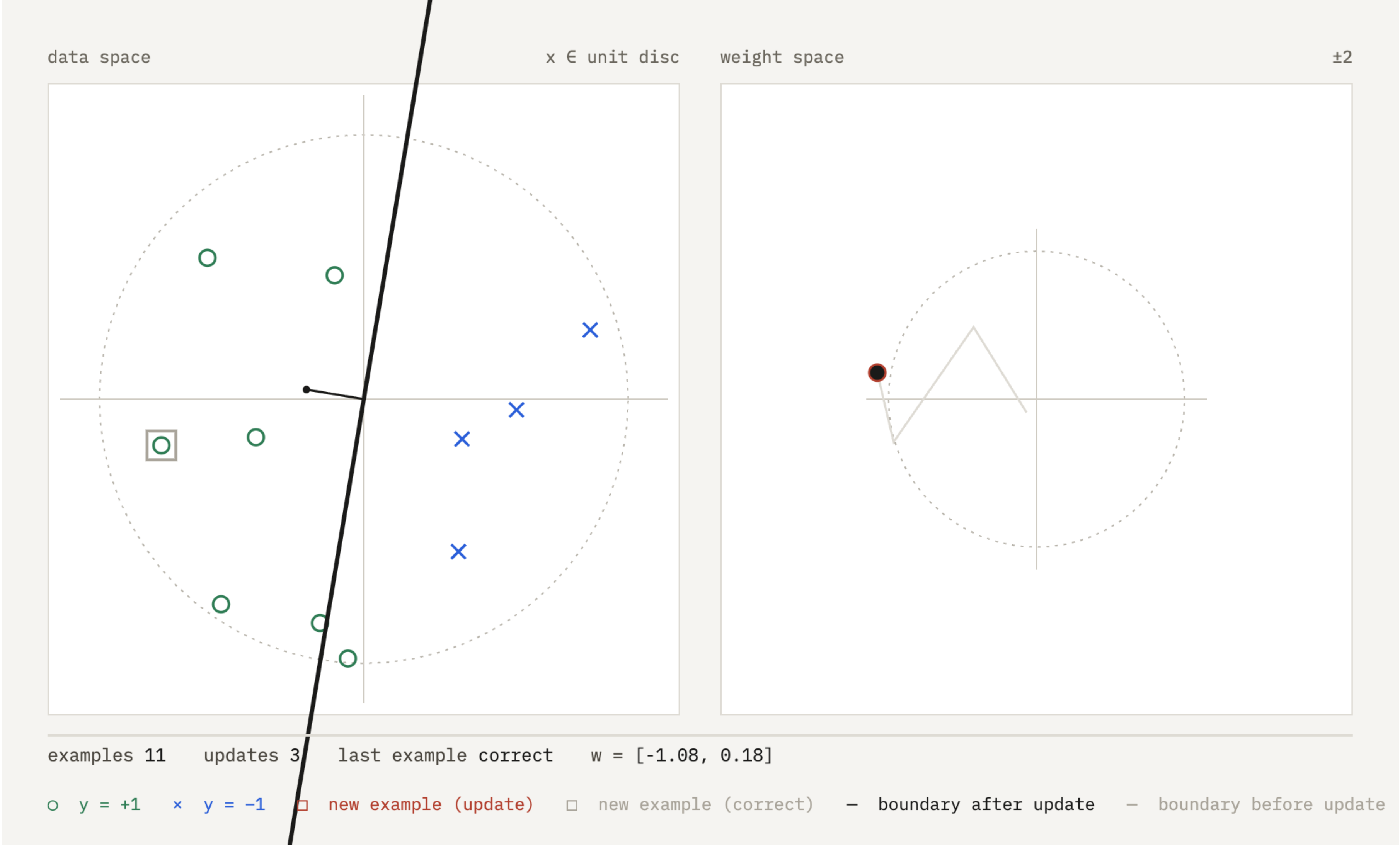
Perceptron



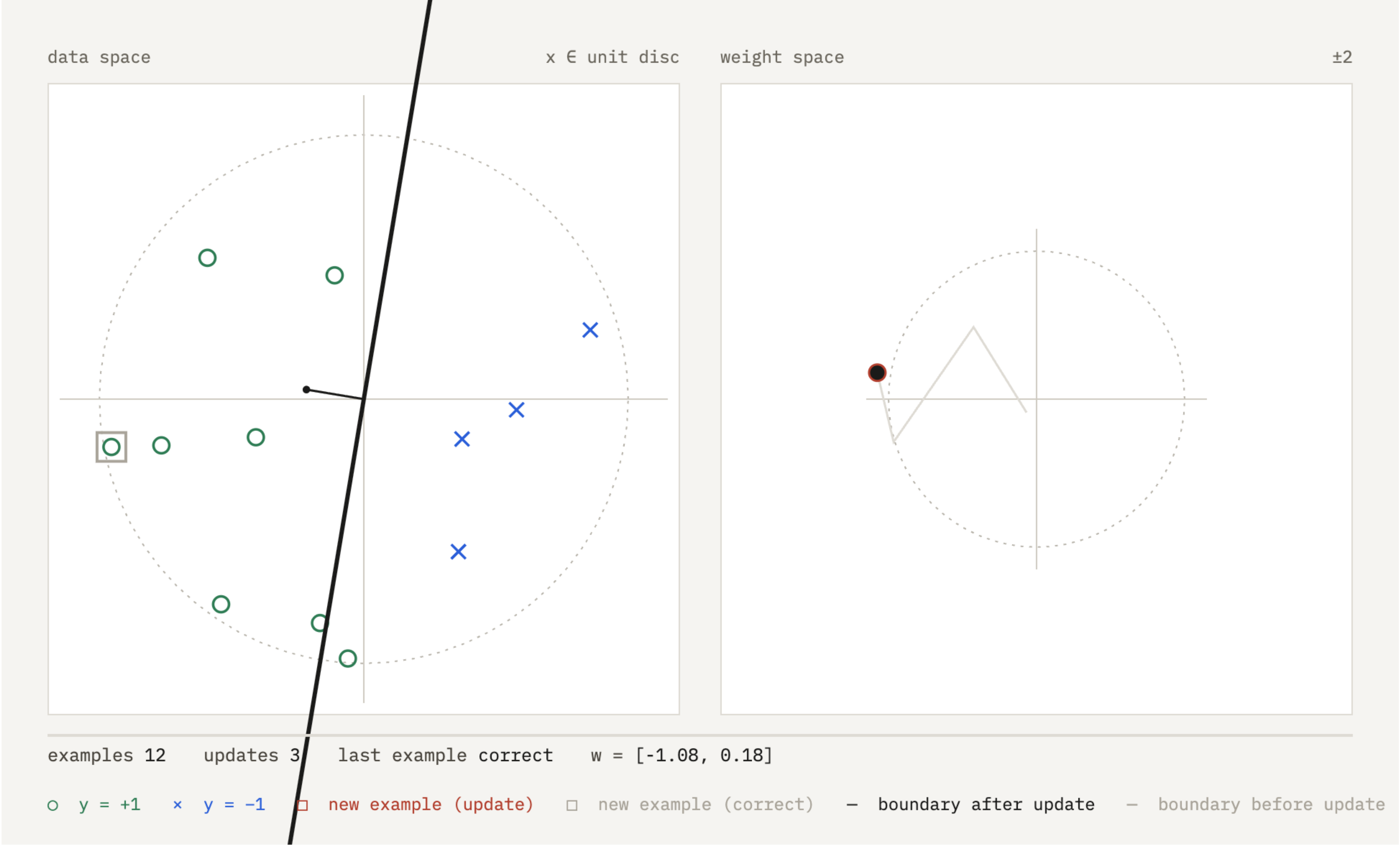
Perceptron



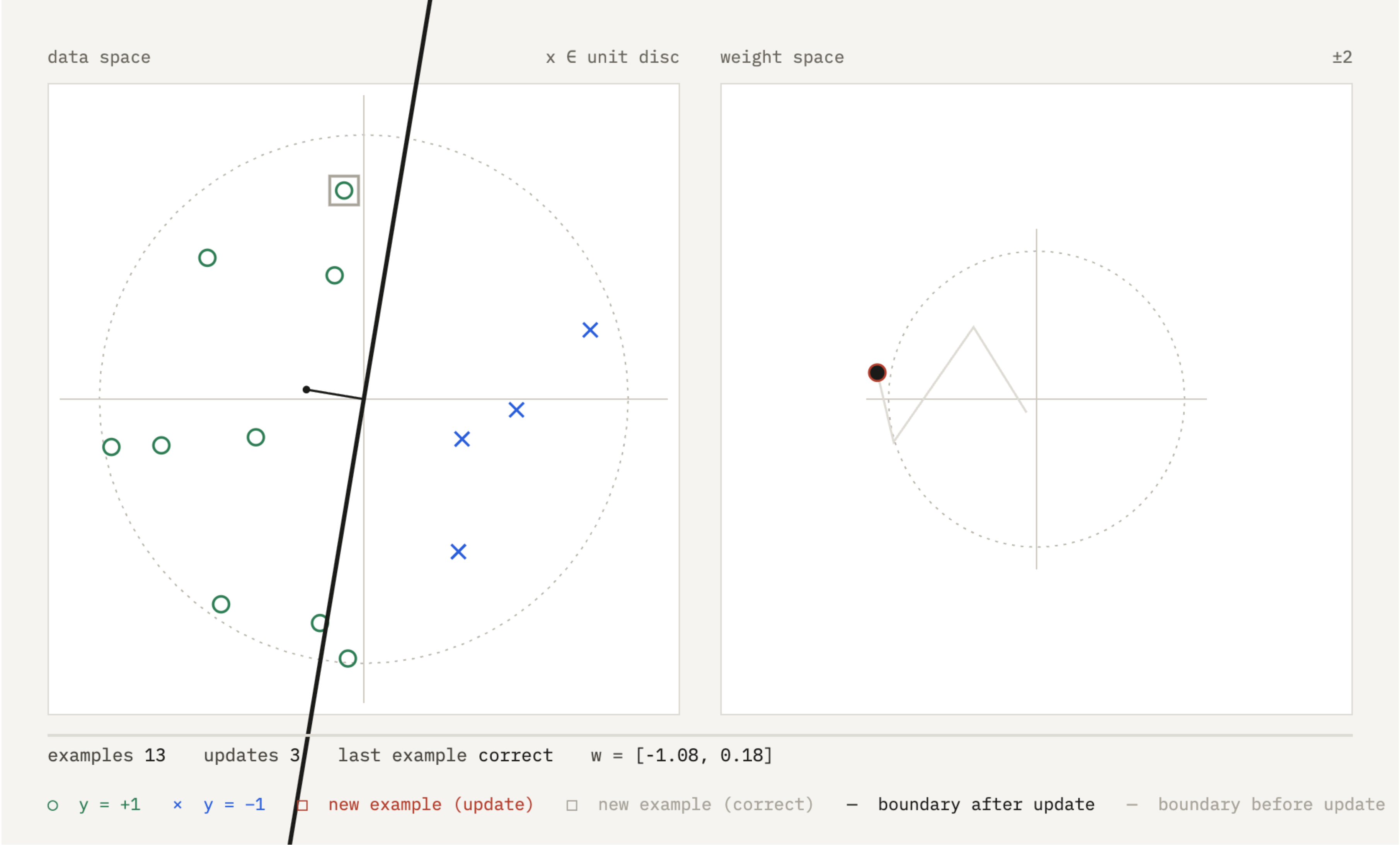
Perceptron



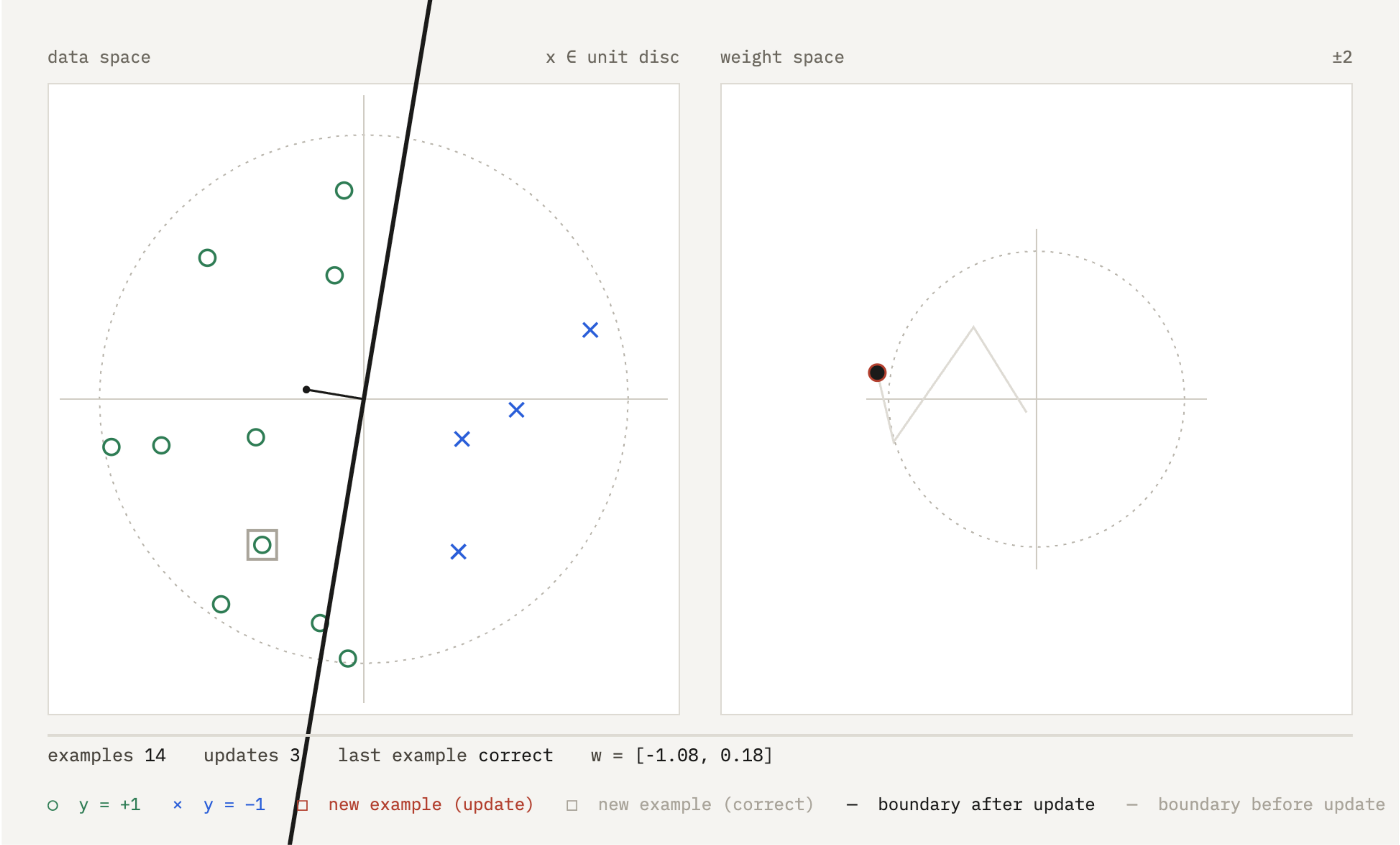
Perceptron



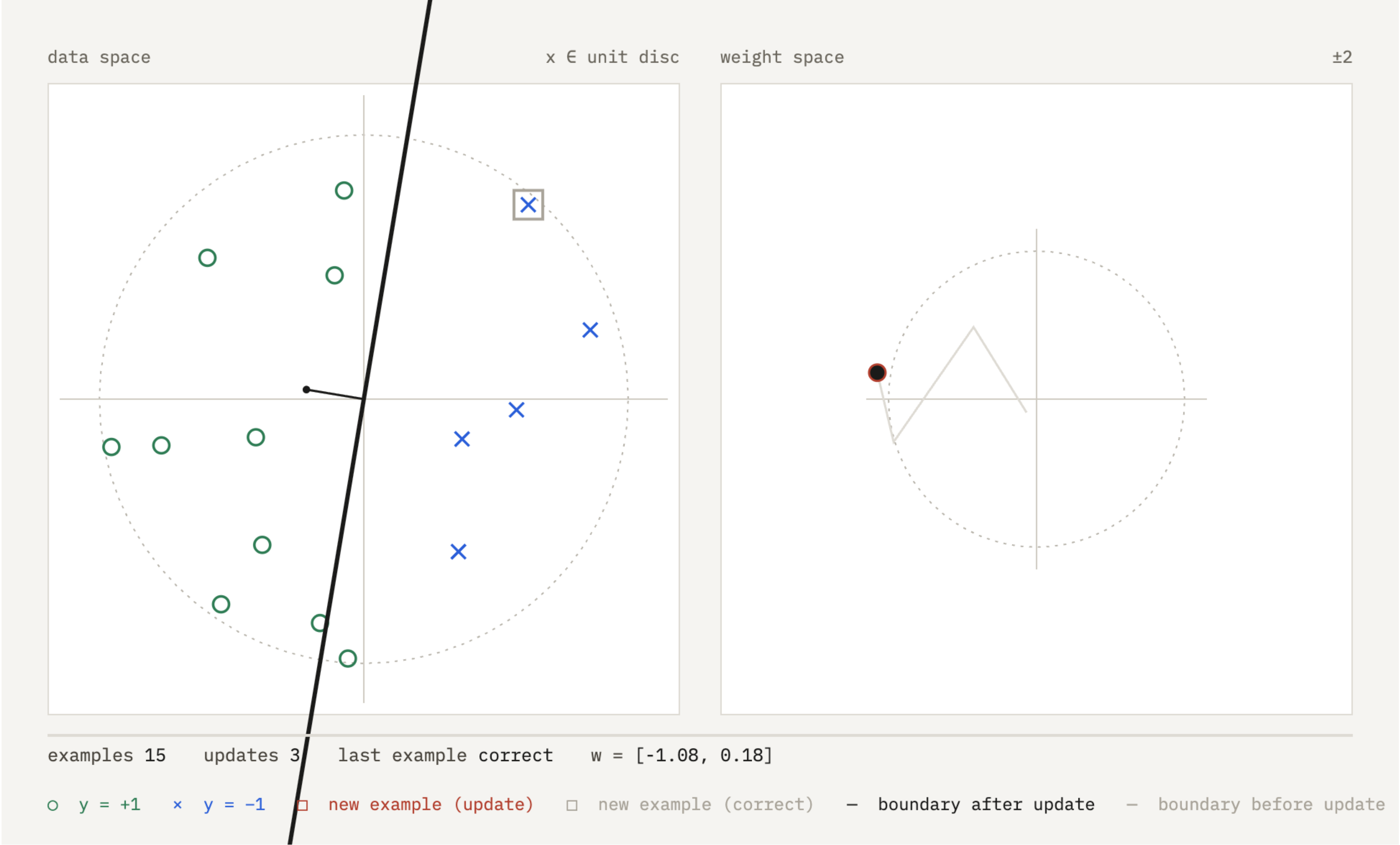
Perceptron



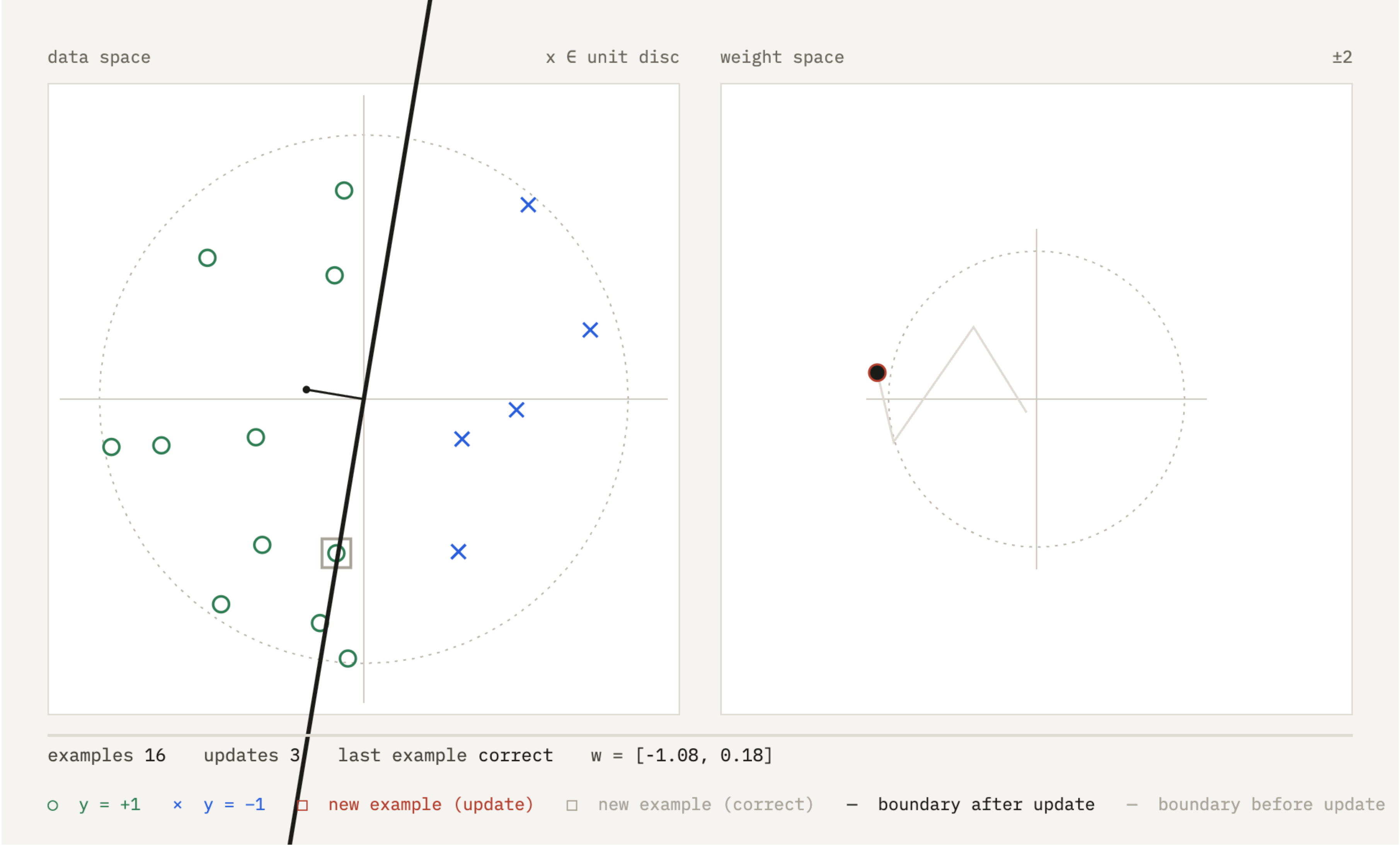
Perceptron



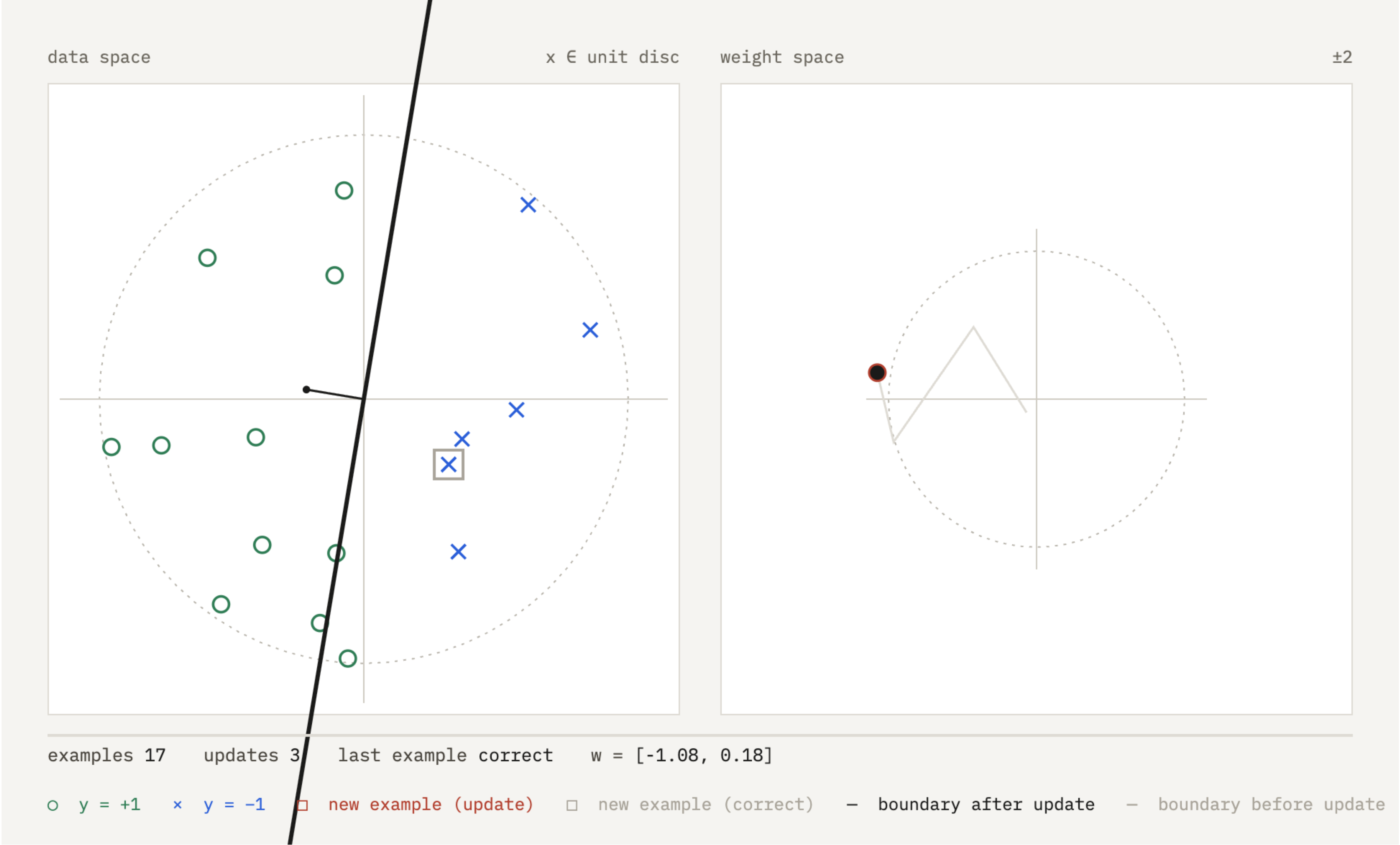
Perceptron



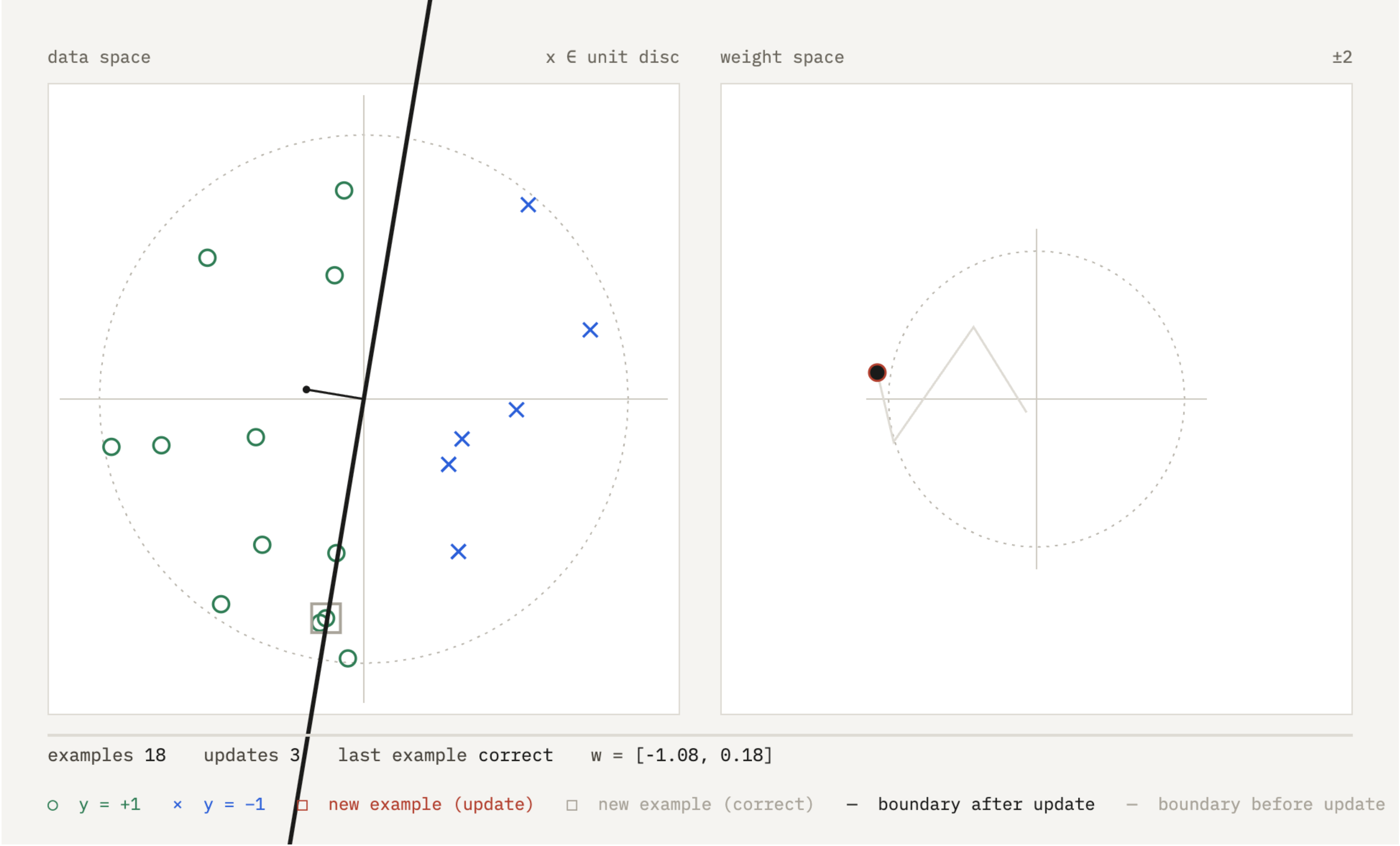
Perceptron



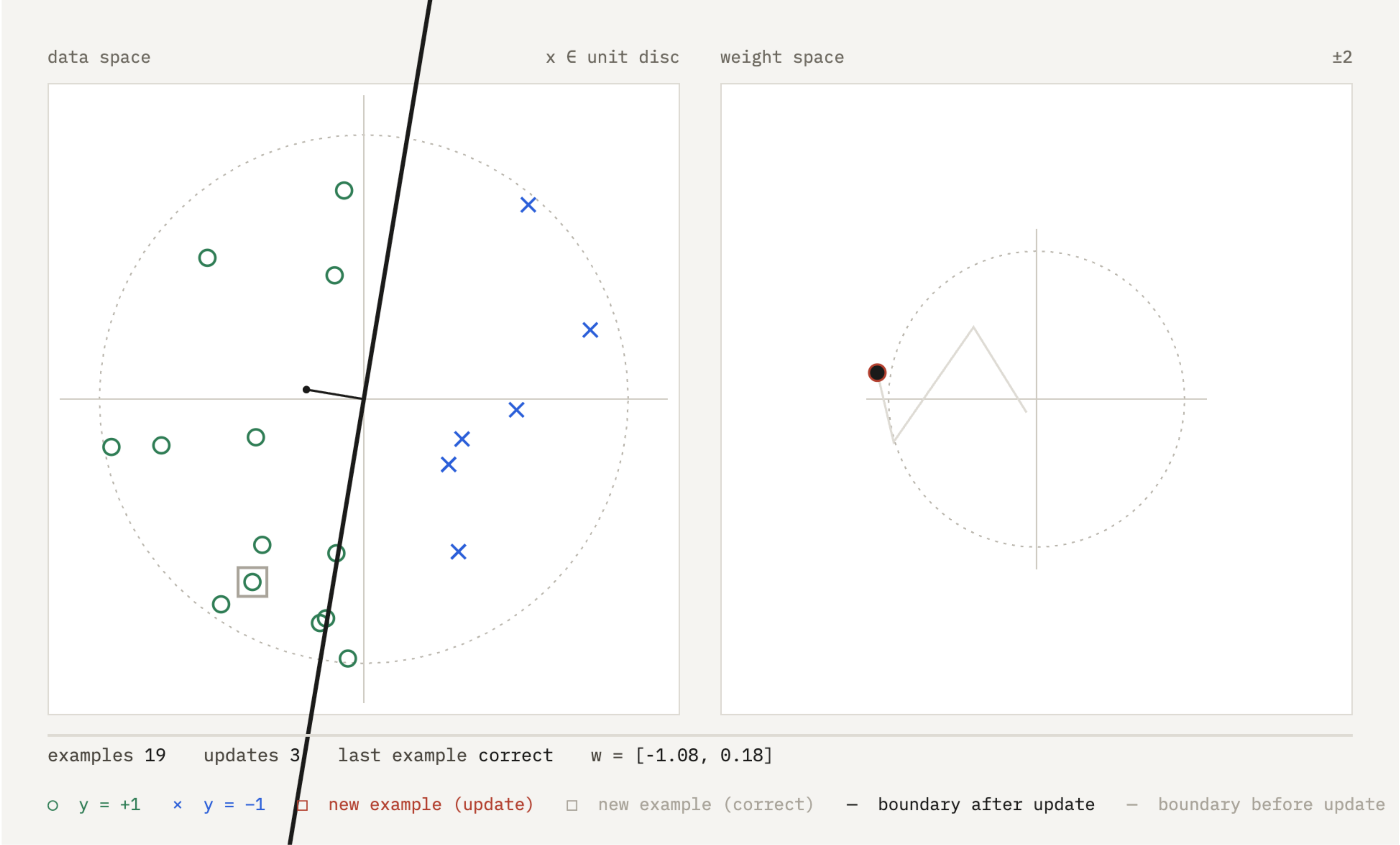
Perceptron



Perceptron



Perceptron



Perceptron

